



Фото: Air Canada Boeing 787 · Wikimedia · CC-BY-SA

Чат-бот выдумал политику — платит компания.

Moffatt v. Air Canada · трибунал ВС · 14.02.2024

- 1 Пассажир спросил чат-бота про тариф по случаю утраты близкого
- 2 Бот: «купи по полной цене, верни разницу в течение 90 дней»
- 3 Реальная политика этого не допускала — и была на той же странице, на которую бот ссылался
- 4 Трибунал: «бот — не отдельное юр. лицо» → компания компенсирует разницу^{1,2}

Это не сбой модели — это неправильный выбор архитектуры под задачу. Об этом классе ошибок вся лекция.

^[1] McCarthy Tétrault — Moffatt v. Air Canada (BC CR 1, 14.02.2024) · ^[2] ABA Business Law Today — компания omg отвечает за AI-чат-бота

1 / 40

Четырнадцатого февраля 2024 года Гражданский трибунал по разрешению споров провинции Британская Колумбия вынес решение по делу *Moffatt v. Air Canada*. История проста. Пассажир Джейк Моффатт зашёл на сайт авиакомпании после смерти бабушки, спросил чат-бота про тариф по случаю утраты близкого и получил ответ: можно купить билет по обычной цене, а затем в течение 90 дней подать заявку на возврат разницы. Это было неправдой — реальная политика Air Canada не допускала ретроактивного возврата по такому тарифу, и об этом было написано на той же странице сайта, на которую чат-бот даже ссылался. Моффатт купил билеты, подал заявку, получил отказ, и дело дошло до трибунала. Авиакомпания защищалась аргументом, что чат-бот — это «отдельное юридическое лицо, отвечающее за свои действия».[1,2] Трибунал этот аргумент отверг и обязал Air Canada компенсировать пассажиру разницу.

Нас в этой истории интересует не юридическая сторона, а инженерная. Задача была элементарной: показать пользователю фиксированную, заранее известную политику тарифа. Для такой задачи существует архитектура, которая работает идеально и стоит почти ничего: статическая страница или детерминированный lookup в таблице правил. Air Canada выбрала вместо этого генеративный чат-бот — архитектуру, которая принципиально способна сочинить несуществующую политику, потому что её работа — генерировать правдоподобный текст, а не извлекать проверенный факт.

Это был не сбой модели. Это был неправильный выбор архитектуры под задачу. И именно про этот класс ошибок — про выбор архитектуры — вся эта лекция. Прежде чем идти дальше, задайте себе вопрос: если задача всего лишь «узнать правило тарифа», какая архитектура её закрывает — и нужен ли здесь ИИ вообще? К этому ответу мы вернёмся в разделе про RAG.

Источники:

[1] McCarthy Tétrault — Moffatt v. Air Canada (BC CRT, 14.02.2024) — бот выдумал политику возврата; трибунал: компания отвечает за ответ бота.
<https://www.mccarthy.ca/en/insights/blogs/techlex/moffatt-v-air-canada-misrepresentation-ai-chatbot>

[2] ABA Business Law Today — компании отвечают за AI-чат-бота — «бот — не отдельное юр. лицо»: неправильный выбор архитектуры под задачу lookup.
https://www.americanbar.org/groups/business_law/resources/business-law-today/2024-february/bc-tribunal-confirms-companies-remain-liable-information-provided-ai-chatbot/

ЛЕКЦИЯ 3

Архитектуры AI-систем: агенты, RAG, API

Какую архитектуру выбрать под задачу —
и когда правильный ответ «не ИИ»

3 курс ИУБ · Модуль 1, обзорная лекция

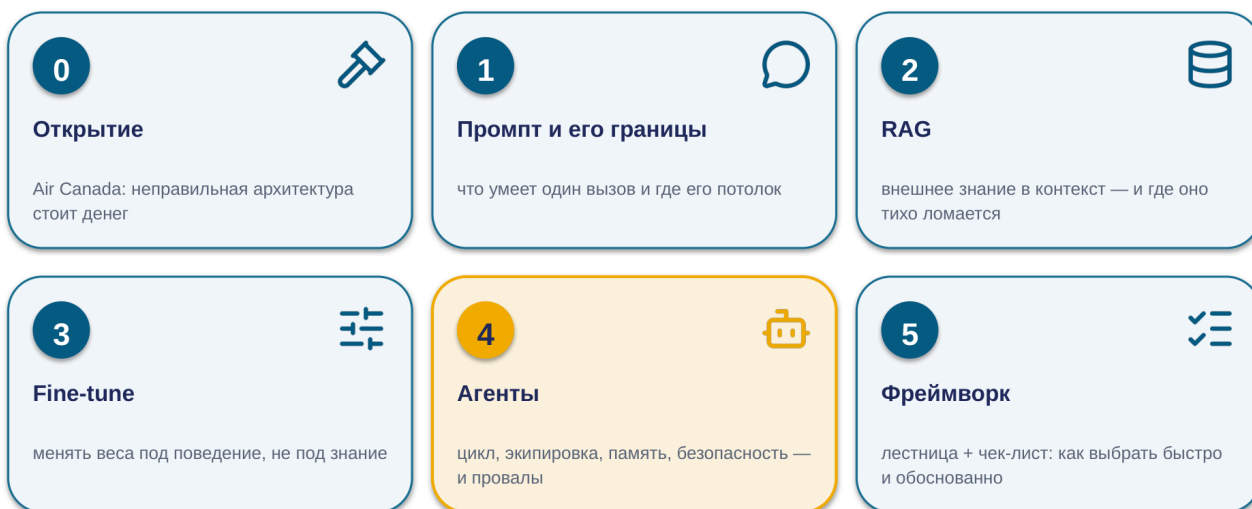
2 / 40

Тема сегодня — архитектуры AI-систем: агенты, RAG, API. Это последняя обзорная лекция вводного модуля курса. Лекция 1 объяснила, что такое AI-модель и как устроен промпт; Лекция 2 — почему промпт работает изнутри: токенизация, эмбединги, внимание, сэмплинг. Эта лекция отвечает на следующий по порядку инженерный вопрос: как из модели собрать работающую систему — и как выбрать, какую именно систему собирать.

Несущая мысль всей лекции одна, и её стоит держать в голове до самого конца: выбор архитектуры — это инженерное решение под конкретную задачу, а не следование моде. Часто правильный ответ — самая простая из возможных архитектур. А иногда правильный ответ — вообще не использовать ИИ. Мы будем возвращаться к этой мысли в каждом разделе: сначала разберём, что умеет один вызов модели и где его границы, затем поднимемся по ступеням сложности — RAG, fine-tuning, агенты — и на каждой ступени зададим один и тот же вопрос: а нужна ли здесь эта сложность вообще, и чем мы за неё платим.

Маршрут лекции — шесть разделов.

Одна несущая линия: выбор архитектуры под задачу — и когда правильный ответ «не ИИ».



3 / 40

Прежде чем углубляться в отдельные архитектуры, полезно увидеть маршрут целиком. Лекция состоит из шести разделов, и они выстроены не как каталог технологий, а как единая линия рассуждения. Раздел ноль — открытие: дело Air Canada, на котором видно, что цена неправильного выбора архитектуры вполне реальна. Первый раздел — промпт и его границы: самый дешёвый и предсказуемый способ применить модель, один вызов, и то, где он упирается в предел. Второй — RAG, поиск-дополненная генерация, надстройка над семантическим поиском из прошлой лекции. Третий — дообучение модели и его честное место в ряду «промпт против RAG против fine-tuning». Четвёртый и самый большой — агенты: как модель становится компонентом системы через API и MCP, из чего собран агент-помощник, где живёт его память и как всё это ломается. И пятый — сборка: лестница архитектурной сложности, маршрут выбора и короткий чек-лист, который можно приложить к новой задаче за пару минут.

Через все шесть разделов проходит одна несущая линия, ради которой стоит держать эту карту перед глазами. Это не «изучите шесть технологий», а «научитесь выбирать, какую из них применять под конкретную задачу — и распознавать случаи, где правильный ответ вообще не использовать ИИ». Каждый раздел заканчивается одинаково: не только «когда эту архитектуру применять», но и «когда её применять не следует». Именно это умение говорить обоснованное «нет» неподходящему инструменту, а не восторг перед самым мощным, и есть то, чему учит лекция.

Что мы переносим из Лекции 2 — и что надстроим.

Вокруг одного вызова модели надстраиваются 4 обязанности. Два готовых блока из Лекции 2 не переобъясняем.



Из Лекции 2 берём готовыми: single-shot инференс и семантический поиск на эмбедингах (основа RAG). Детали каждой обязанности — дальше по разделам.

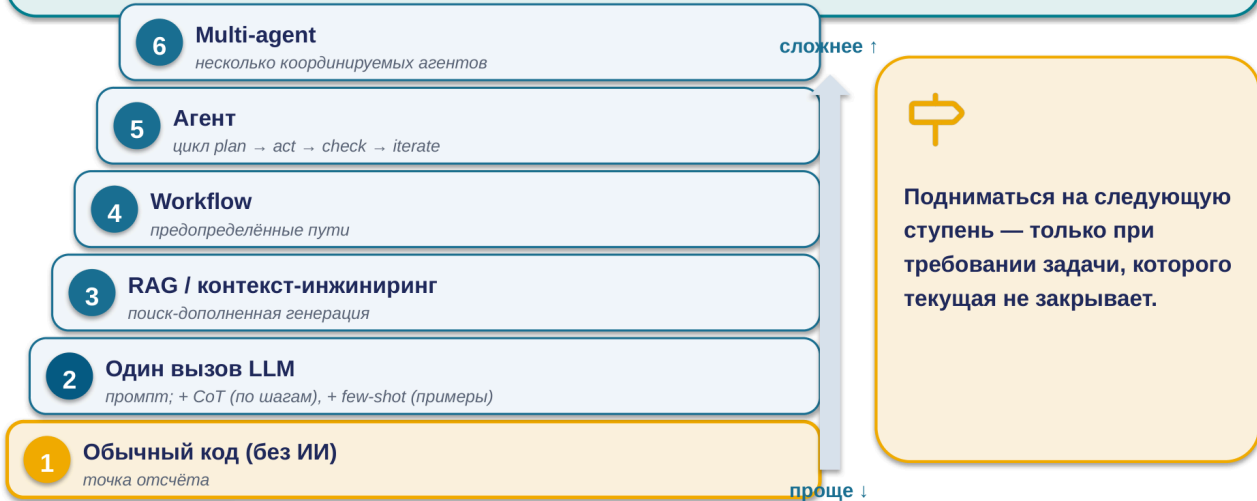
4 / 40

Из Лекции 2 мы переносим сюда два готовых блока и не будем их переобъяснять. Первый — семантический поиск на эмбедингах из Лекции 2, раздел 2: текст превращается в вектор, близость векторов означает близость смыслов, поэтому искать по смыслу можно без точного совпадения слов. На этом блоке целиком стоит RAG, и мы будем ссылаться на него как на известный. Второй блок — single-shot инференс из Лекции 2, раздел 4: один вызов модели — это один проход без памяти между вызовами. Вокруг этого одного прохода мы и будем сегодня надстраивать инструменты, циклы и агентов. Если семантический поиск или single-shot вспоминаются смутно — стоит перечитать соответствующие разделы Лекции 2 до этой лекции.

Картинка на слайде задаёт структуру всей лекции. В центре — один вызов модели, самая базовая архитектура. Вокруг него — четыре обязанности, которые мы разберём. Первая — RAG: это и есть тот самый семантический поиск из Лекции 2 плюс генерация поверх извлечённого; стрелка от эмбедингов к RAG-узлу подчёркивает, что новый материал здесь надстраивается на уже знакомый, а не вводится с нуля. Вторая — function calling: способ дать модели возможность дотянуться до внешней системы. Третья — MCP: стандарт, которым инструменты подключаются к модели. Четвёртая — цикл агента: много шагов вокруг одного прохода. К концу лекции эти четыре обязанности будут для вас не списком модных слов, а набором инструментов с известными границами применимости, между которыми вы умеете осознанно выбирать. Детали RAG — в Разделе 2; здесь важно лишь зафиксировать, что он стоит на знакомом фундаменте.

ЦЕНТРАЛЬНЫЙ ВОПРОС ЛЕКЦИИ

У меня есть задача и доступ к LLM. Какую архитектуру выбрать — и когда правильный ответ «не ИИ»?



Лестница — карта лекции, не требование понять всё сейчас. Каждую ступень разберём отдельно.

5 / 40

Это центральный вопрос всей лекции, к которому мы будем возвращаться в каждом разделе: у меня есть задача и доступ к LLM — какую архитектуру выбрать, и когда правильный ответ «не ИИ»? Обратите внимание на вторую половину вопроса. Она не риторическая. Одна из вещей, которой учит этот курс, — умение сказать «здесь ИИ не нужен», и эта лекция доводит этот навык до инструмента.

Ответом будет лестница из шести ступеней — она на слайде. Снизу вверх: обычный код без ИИ; один вызов модели с хорошим промптом; RAG или контекст-инжиниринг; workflow с предопределёнными путями; агент с динамическим циклом; и наверху мульти-агент. Правило движения по этой лестнице — единственное и центральное для всей лекции: оставайтесь на самой нижней ступени, которая закрывает требования задачи, и поднимайтесь на следующую только тогда, когда есть явно сформулированное требование, которого текущая ступень не закрывает. Каждый подъём оплачивается новыми режимами отказа, стоимостью, латентностью, ухудшением аудируемости и новой поверхностью атаки.

Заметьте, что нижняя ступень — обычный код без ИИ. Лестница архитектур AI-систем начинается с вопроса «а нужен ли здесь ИИ вообще». Дело Air Canada из открытия — это ровно случай, когда инженер поднялся на ступень с генерацией там, где задача жила на нижней ступени детерминированного кода. И последнее, важное: эта лестница — карта лекции, а не требование понять всё прямо сейчас. Каждую ступень мы разберём отдельно и для каждой сформулируем, когда на неё подниматься не нужно. Сейчас достаточно запомнить только структуру и правило.

РАЗДЕЛ 1

Промпт и его границы

Лестницу мы увидели целиком — теперь снизу: что умеет один вызов и где его потолок, прежде чем что-либо усложнять.



Раздел 0
Открытие

Раздел 1
Промпт

Раздел 2
RAG

Раздел 3
Fine-tune

Раздел 4
Агенты

Раздел 5
Фреймворк

6 / 40

Мы только что поставили центральный вопрос лекции: есть задача и доступ к языковой модели — какую архитектуру выбрать, и когда правильный ответ «не ИИ». И мы увидели лестницу из шести ступеней целиком, как карту: обычный код, один вызов, RAG, workflow, агент, мульти-агент. Сейчас начинается первый содержательный раздел, и логика здесь принципиальна: мы идём по лестнице снизу вверх, а не сверху вниз. Не «какой самый мощный инструмент существует», а «достаточно ли самого простого, и если нет — то под какое именно требование задачи мы поднимаемся на ступень выше».

Самая нижняя ступень с участием модели — это один вызов с хорошим промптом. Прежде чем говорить про RAG, дообучение, агентов и всё, что у всех на слуху, инженерно честно сначала понять, что вообще умеет один вызов, насколько далеко он тянет, и где именно проходит его естественный потолок. Потому что подавляющее большинство задач закрывается именно здесь, дёшево и предсказуемо, а каждый подъём по лестнице оплачивается ростом стоимости, латентности и числа способов всё сломать. В этом разделе мы разберём четыре вещи: почему дефолт — это один вызов; что такое Chain-of-thought и где он помогает; почему проговорённое моделью рассуждение нельзя принимать за её настоящую причину; и как курировать контекст, а не сваливать в него всё подряд. Это и есть граница, за которой начинается следующая ступень.

Дефолт — один вызов с хорошим промптом.

Модель знает только то, что в промпте (плюс то, что было в весах на обучении). Больше ей взять неоткуда.



Один вызов LLM с хорошим промптом

- минимальная стоимость (один проход)
- минимальная латентность (нет лишних обращений)
- максимальная предсказуемость (нет петель, нет retrieval, который тихо деградирует)

Добавляешь RAG →

конвейер индексации + векторное хранилище + компонент retrieval (может молча деградировать) + метрики его качества

Добавляешь инструменты / цикл →

внешние вызовы (падают, тормозят) + петли (расходятся) + недетерминизм траектории + новая поверхность атаки

Не усложняй архитектуру без причины, выраженной в требованиях задачи. Это распределение бремени доказательства, а не примитивизм.

[1] Anthropic — Building Effective Agents («найди простейшее»)

7 / 40

Прежде чем выбирать между RAG, fine-tuning и агентами, зафиксируем точку отсчёта. Самая дешёвая, надёжная и предсказуемая AI-архитектура — это один вызов модели с хорошо составленным промптом[1]. Один вызов — в Лекции 2 это single-shot инференс — не имеет ни внешнего поиска, ни цикла, ни инструментов: на вход промпт, на выход ответ, памяти между вызовами нет. У него минимальная стоимость, минимальная латентность и максимальная предсказуемость: нет петель, которые расходятся, нет retrieval, который тихо деградирует.

Зафиксируем границу, несущую для всего раздела: модель знает только то, что попало в промпт, плюс то, что осело в её весах при обучении. Это единственный источник информации, доступный одному вызову. Всё, чего нет ни в промпте, ни в весах, модели неизвестно, и тогда она либо честно откажется отвечать, либо сгенерирует правдоподобный, но ни на что не опирающийся текст. Именно эта граница — причина, по которой дальше появляется RAG, расширяющий то, что «в промпте», и fine-tuning, расширяющий то, что «в весах».

Инженерное правило по умолчанию: не усложнять архитектуру без причины, выраженной в требованиях задачи. Это не призыв к примитивизму, а распределение бремени доказательства. По умолчанию архитектура — один вызов. Любой переход выше требует явного обоснования: вот требование задачи, которое одним вызовом не закрывается, поэтому я добавляю RAG, инструмент или цикл. Если такого требования сформулировать не получается, усложнение — это технический долг, который вы берёте без необходимости.

Полезно зафиксировать, во что обходится каждый подъём, чтобы бремя доказательства было перечнем. Добавляя RAG, вы добавляете конвейер

индексации, векторное хранилище, компонент retrieval, который может молча деградировать, и метрики его качества. Добавляя инструменты и цикл агента — внешние вызовы, которые падают и тормозят, петли, которые расходятся, недетерминизм траектории и новую поверхность атаки. Ни один из этих пунктов не появляется у одного вызова. Поэтому «один вызов по умолчанию» — не консерватизм, а отказ оплачивать инфраструктуру и режимы отказа, которых задача не требует.

Источники:

[1] Anthropic — Building Effective Agents («найди простейшее») — не усложняй архитектуру без требования задачи — распределение бремени доказательства.
<https://www.anthropic.com/research/building-effective-agents>

Роль в промпте настраивает тон — не точность.

«Ты — опытный юрист» сдвигает внимание модели (механизм — Лекция 2) в сторону текста, похожего на ответ такой роли, — но это про стиль, не про факты.

Расхожий миф

«Напишу роль эксперта — и модель будет точнее отвечать по фактам»

Что показал эксперимент

162 персоны · 6 типов отношений · 8 доменов · 2410 вопросов фактического QA · 4 семейства LLM. Персоны НЕ улучшили точность ответа по сравнению с ответом без персоны вообще.



На что роль влияет реально

на тон и глубину изложения — насколько формально и подробно, а не верен ли факт (эффект модель-специфичен).

Нужна фактическая точность? Инструмент — не формулировка роли, а грамотный контекст и, при необходимости, RAG (опора на проверяемый источник). Это ещё один пункт «магической пилюли», опровергнутый измерением.

[1] Zheng et al. 2024, Findings of EMNLP — персоны не улучшают точность фактов · Zheng, Pei, Logeswaran, Lee, Jurgens · Findings of EMNLP 2024 (arXiv:2311.10054) + arXiv:2605.29420 (2026).

8 / 40

Формула промпта «роль плюс задача плюс контекст» знакома с первой лекции, и роль — самый частый первый элемент, который инженер пишет в системный промпт: «ты опытный юрист», «ты строгий редактор». Интуиция обычно такая: чем точнее роль, тем компетентнее ответ, будто модель переключается в режим эксперта. Это один из самых живучих мифов, и его стоит разобрать честно. Механически роль — это токены, которые появляются в контексте раньше вопроса и участвуют во всех вычислениях внимания, о котором шла речь в прошлой лекции: она не включает экспертный режим тумблером, а сдвигает распределение вероятностей следующих токенов в сторону текста, похожего на то, как отвечал бы держатель этой роли в обучающих данных.

Ключевой вопрос — сдвигает ли это фактическую точность[1], а не только подачу. Специально спроектированное исследование Zheng и коллег дало однозначный ответ: 162 персоны, восемь доменов экспертизы, 2410 фактических вопросов, четыре семейства моделей — и персоны не улучшают точность по сравнению с ответом вообще без роли. Модель, которой сказали «ты нобелевский лауреат по физике», не отвечает на физику точнее модели без роли. Отдельное исследование показывает, на что роль влияет реально: на тон и глубину изложения — насколько формально и подробно раскрыта тема, — но не на то, верен ли сообщаемый факт, и этот эффект модель-специфичен. Вывод для инженера: роль — инструмент настройки стиля, а не повышения точности. Нужна фактическая точность — работает грамотный контекст и, при необходимости, опора на проверяемый источник, а не удачная формулировка роли.

Источники:

[1] Zheng et al. 2024, Findings of EMNLP — персоны не улучшают точность фактов

— 162 персоны · 8 доменов · 2410 фактических вопросов · 4 семейства → нет прироста точности. <https://arxiv.org/abs/2311.10054>

Структура промпта: разделяй инструкцию, контекст, данные.

Плоский промпт заставляет модель угадывать, где кончается инструкция и начинаются данные. Явная граница снимает эту неоднозначность.



Инструкция

что сделать



Контекст

на основе чего



Данные

с чем именно работать

Разделители (delimiters)

- XML-теги: `<инструкция>...</инструкция>`
- Markdown-заголовки: `## Задача · ## Данные`
- Тройные кавычки / бэктики вокруг данных

Тот же принцип, что structured output (Раздел 4): там схема задаёт структуру ВЫХОДА, здесь разделители — структуру ВХОДА. Ясная структура вместо «выведи форму по смыслу».

Модель, которой показали, где кончается инструкция и начинаются данные, реже принимает фрагмент данных за новую команду — та же путаница лежит в основе prompt injection (Раздел 4).

9 / 40

Если роль отвечает за тон, то за качество ответа в куда большей степени отвечает структура промпта — насколько ясно в тексте запроса разделены три разных вида содержимого: инструкция, то есть что сделать; контекст, то есть на основе чего; и данные, то есть с чем именно работать. Плоский промпт, где всё это слито в один абзац свободного текста, заставляет модель самостоятельно угадывать границы между этими ролями, и чем длиннее и разнороднее промпт, тем выше цена такой неоднозначности.

Практический инструмент — разделители: явные текстовые маркеры, отделяющие один вид содержимого от другого. Распространённые варианты — XML-подобные теги, markdown-заголовки или тройные кавычки вокруг вставляемого текста. Конкретный синтаксис менее важен, чем сам факт разделения: модель, которой явно показали, где кончается инструкция и начинаются данные для обработки, реже путает одно с другим. В частности, реже воспринимает фрагмент вставленных данных как новую команду — а это та же путаница «данные против команды», которая лежит в основе prompt injection, атаки, которую мы подробно разберём в разделе про безопасность агентов.

Это тот же инженерный принцип, что мы встретим у structured output: там разработчик задаёт модели строгую схему для выхода, здесь — структуру для входа. Механизм разный, а идея одна: модель работает лучше, когда её ясно ориентируют, что есть что, а не оставляют выводиться это по смыслу текста.

Chain-of-thought помогает — но его нельзя аудировать.

Без CoT

«Было 23 яблока, 7 испортились, докупили 2 ящика по 6.
Сколько хороших?»

→ правдоподобное, но неверное число

С CoT («решай по шагам»)

$23 - 7 = 16$ · $2 \times 6 = 12$

$16 + 12 = 28$

→ верно

Технически это всё ещё один вызов — CoT инструмент под класс задач (цепочка шагов), не глобальный тумблер.

Но проговорённое рассуждение не обязано отражать реальную причину ответа (низкая faithfulness):

~25%

Claude 3.7 Sonnet

~39%

DeepSeek R1

— доля случаев, где модель упомянула реально использованную подсказку.

Контроль на самообъяснении модели — не контроль. Человек-валидатор проверяет РЕЗУЛЬТАТ и факты против внешнего источника, а не правдоподобность текста. И это же — почему шаг check в цикле агента (Раздел 4) не должен быть самооценкой модели.

[1] Wei et al. 2022 — Chain-of-Thought Prompting · [2] Anthropic — Reasoning Models Don't Always Say What They Think · Anthropic, апрель 2025 - верность падает на трудных задачах · перепроверить ко дню лекции.

10 / 40

Chain-of-thought, или пошаговое рассуждение, — это приём, при котором модель просят не выдавать ответ сразу, а сначала пошагово рассуждать вслух, и только затем формулировать итог. Технически это всё ещё один вызов: изменён только промпт и форма ответа. Посмотрите на рабочий пример про яблоки: было 23, семь испортились, докупили два ящика по шесть. Без пошагового рассуждения модель часто выдаёт правдоподобное, но неверное число, потому что сэмплинг генерирует ответ потоково и короткий путь легко уводит в ошибку. С CoT модель порождает промежуточные вычисления — 23 минус 7 равно 16; два по шесть это 12; 16 плюс 12 равно 28[1] — и качество на арифметике и многошаговой логике заметно растёт, потому что каждый сгенерированный токен служит опорой для внимания на следующем шаге. Но «CoT почти всегда улучшает» — заблуждение: на прямом извлечении факта или простой классификации рассуждение не помогает и может навредить — удлиняет ответ и уводит в правдоподобную ошибку. Это инструмент под класс задач, а не тумблер «сделать лучше».

Теперь предел этого приёма, и он важнее самого приёма. Естественно предположить, что раз модель рассуждает вслух, по этому рассуждению можно проверить, как она пришла к ответу. Это неверно фундаментально. Faithfulness, верность рассуждения, — степень, в которой проговорённая цепочка отражает реальные факторы, повлиявшие на ответ. Anthropic измерила это напрямую: модели давали задачу с подсказкой, меняющей ответ, и смотрели, упоминает ли модель эту подсказку в рассуждении. Claude 3.7 упоминал реально использованную подсказку примерно в четверти случаев, DeepSeek R1 — примерно в двух из пяти[2]; в остальных модель меняла ответ под влиянием подсказки, но строила выдуманную аргументацию. Вывод структурный: цепочка

— сгенерированный текст, подчиняющийся той же механике сэмплинга, произведённый быть правдоподобным, а не верным протоколом внутреннего вычисления. Unfaithfulness — свойство класса, не баг конкретной модели. Практический вывод и одна из сквозных линий курса: человек-валидатор проверяет результат и факты против независимого источника, а не самообъяснение модели.

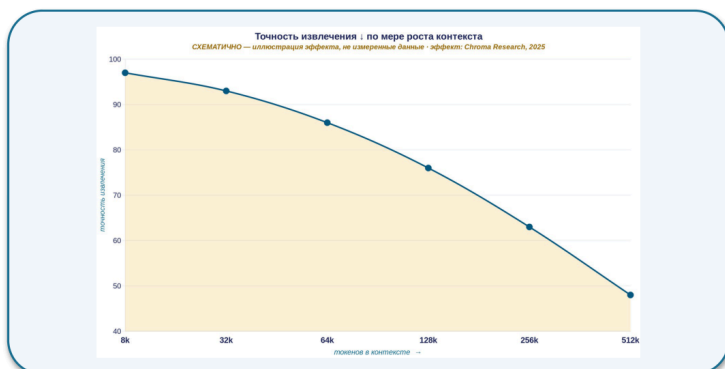
Источники:

[1] Wei et al. 2022 — Chain-of-Thought Prompting — пошаговое рассуждение поднимает надёжность на арифметике/многошаговой логике.
<https://arxiv.org/abs/2201.11903>

[2] Anthropic — Reasoning Models Don't Always Say What They Think — faithfulness: Claude 3.7 ~25%, DeepSeek R1 ~39% — рассуждение не обязано отражать причину. <https://www.anthropic.com/research/reasoning-models-dont-say-think>
[VFY-day-of]

Контекст-инжиниринг: минимум высокосигнального.

Промпт-инжиниринг — одна инструкция. Контекст-инжиниринг — курирование всего набора токенов, видимых модели на инференсе.



context rot = тот же «lost in the middle» из L2 — новый термин, не новая сущность

Когда НЕ RAG (точка 1)

малый стабильный корпус, влезает в окно → full-context + кэширование префикса, а не RAG-инфраструктура



RAG здесь добавил бы хрупкость без выигрыша

«Найти наименьший набор высокосигнальных токенов, максимизирующий вероятность желаемого исхода» — это инженерное требование, не эстетика».

[1] Chroma Research — Context Rot · [2] Liu et al. 2023 — Lost in the Middle · [3] Anthropic — Effective Context Engineering

11 / 40

Если chain-of-thought — это про форму одного вызова, то контекст-инжиниринг — про его содержимое. Введём различие. Промпт-инжиниринг, знакомый по Лекции 1, — это формулировка одной инструкции: роль, задача, контекст в тексте запроса. Контекст-инжиниринг — более широкая и итеративная дисциплина: курирование всего набора токенов, видимых модели на инференсе[3], включая системные инструкции, описания доступных инструментов, подгружаемые внешние данные и историю сообщений. Главный принцип формулируется так: найти наименьший набор высокосигнальных токенов, максимизирующий вероятность желаемого исхода.

Почему минимальность контекста — не эстетика, а инженерное требование? Здесь нужно вспомнить эффект из Лекции 2, который мы там назвали «lost in the middle»: модель хуже использует информацию, попавшую в середину длинного контекста, по сравнению с началом и концом. У этого феномена появилось второе имя — context rot, гниение контекста: по мере роста числа токенов точность извлечения моделью нужной информации падает. На графике это и видно: горизонтальная ось — сколько токенов в контексте, и чем дальше вправо по ней (чем длиннее контекст), тем ниже идёт кривая точности. Это тот же самый феномен, что «lost in the middle» из Лекции 2, под новым практическим названием — я делаю эту сшивку явно, чтобы термин не воспринимался как новая сущность. Механизм тот же: попарные связи механизма внимания растут квадратично с длиной.

Прямое следствие: «просто положить всё в контекст» — не стратегия. Большое контекстное окно даёт возможность вместить много текста, но не гарантию, что модель этим текстом воспользуется правильно. И это, забегаая вперёд, один из ключей к вопросу «когда НЕ RAG», который мы разберём в Разделе 2: иногда

правильный ответ — не строить retrieval-инфраструктуру, а аккуратно курировать небольшой стабильный контекст и переиспользовать его через кэширование. То есть это первый практический критерий: если корпус знаний мал, стабилен и помещается в окно — RAG здесь добавил бы хрупкость без выигрыша, и правильный ответ проще, чем кажется.

Источники:

[1] Chroma Research — Context Rot — точность извлечения падает с ростом числа токенов в контексте. <https://research.trychroma.com/context-rot>

[2] Liu et al. 2023 — Lost in the Middle — тот же феномен «lost in the middle» из Лекции 2 — новый термин, не новая сущность. <https://arxiv.org/abs/2307.03172>

[3] Anthropic — Effective Context Engineering — минимальный высокосигнальный контекст — инженерное требование, не эстетика.
<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>

Чит-шит: как строить промпт.

Минимум, ниже которого промпт систематически недорабатывает. Приложите к любому промпту перед первым запуском.

1 Роль

если нужен тон/регистр — и НЕ как обещание точности

2 Задача

конкретное проверяемое действие, не расплывчатое пожелание

3 Контекст

минимально необходимое, не «всё, что могло бы пригодиться»

4 Формат вывода

указан явно, если ответ машинно-обрабатываем

5 Разделители

если содержимого больше одного вида (инструкция/данные)

6 Примеры (few-shot)

только если формат неочевиден из одной инструкции

7 CoT

только если задача требует многошагового рассуждения

8 Длина

не длиннее необходимого — лишние токены «тонут» в контексте

Компактная форма всего раздела: роль ≠ точность, структура помогает разделить входы, CoT — точечный инструмент, контекст — минимальный. Для крупных архитектурных решений (RAG / FT / агент) — восьмишаговый чек-лист Раздела 5.

12 / 40

Соберём весь раздел про промпт в один практический инструмент — короткий чек-лист, который можно приложить к любому промпту перед его первым запуском. Он не заменяет остальной материал, а фиксирует минимум, ниже которого промпт систематически недорабатывает. Первое: роль задана, если нужен конкретный тон или регистр, и явно не используется как обещание точности — если задача требует фактической верности, роль эту задачу не решает. Второе: задача сформулирована как конкретное проверяемое действие, а не расплывчатое пожелание. Третье: контекст содержит минимально необходимое, а не всё, что могло бы пригодиться. Четвёртое: формат вывода указан явно, если ответ должен быть машинно-обрабатываемым.

Пятое: расставлены разделители, если промпт содержит больше одного вида содержимого — инструкцию, контекст, вставляемые данные. Шестое: примеры добавлены только если формат ответа неочевиден из одной инструкции; если модель справляется и без них, примеры лишь расходуют контекст. Седьмое: пошаговое рассуждение включено только там, где задача требует многошаговой логики, а для прямого извлечения факта — не нужно. Восьмое: проверено, что промпт не длиннее необходимого, потому что каждый лишний абзац — это токены, которые нужно оплатить и которые рискуют утонуть в контексте. Этот чек-лист — не процедура ради процедуры, а сжатый свод всего раздела: роль не равна точности, структура помогает разделить входы, рассуждение — точечный инструмент, а контекст — минимальный, а не максимальный.

РАЗДЕЛ 2

RAG: поиск-дополненная генерация

Извлечь релевантное → положить в контекст → ответить с опорой на источник



Раздел 0
Открытие

Раздел 1
Промпт

Раздел 2
RAG

Раздел 3
Fine-tune

Раздел 4
Агенты

Раздел 5
Фреймворк

13 / 40

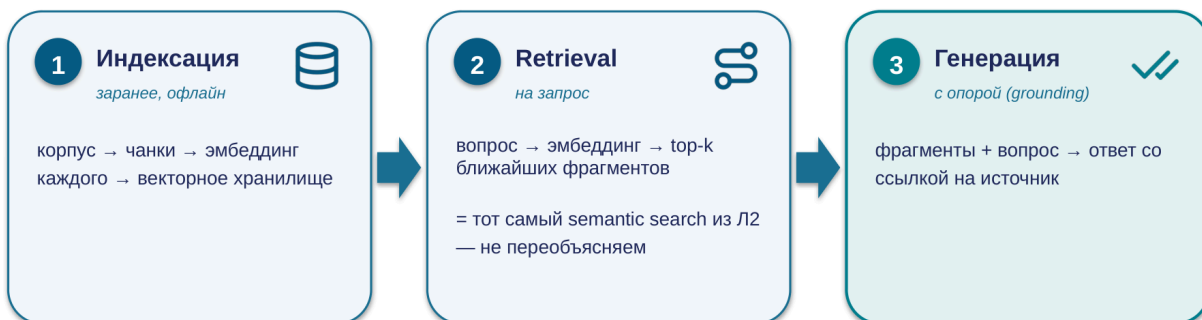
Мы закончили первый раздел: дефолт — один вызов, его можно усилить пошаговым рассуждением, но у рассуждения есть предел верности, и контекст нужно курировать, а не сваливать. Теперь — следующая ступень лестницы.

Один вызов с хорошим промптом упирается в естественную границу: модель знает только то, что попало в её веса при обучении, плюс то, что вы вручную положили в контекст. Если задача требует знаний, которых в весах нет, — например, приватной базы компании, — или которые меняются быстрее, чем выходят новые версии модели, — например, актуальных цен или документов, — один вызов не справится. Здесь начинается RAG: поиск-дополненная генерация.

В одной фразе принцип такой: перед вызовом модели система сначала извлекает релевантные фрагменты из внешнего хранилища знаний, кладёт их в контекст вместе с вопросом, и только затем модель генерирует ответ, опираясь на эти фрагменты. В этом разделе мы разберём: как RAG устроен в три шага и почему механизм поиска в нём — это ровно семантический поиск из Лекции 2; когда RAG — правильный выбор, а когда — нет, и почему второе важнее; и как RAG тихо ломается на масштабе, замыкая раздел обратно на дело Air Canada из открытия.

Принцип RAG — три шага.

RAG = индексация → retrieval (semantic search Л2) → генерация с опорой; «не знаю» — корректный ответ.



«Не знаю» / «см. источник X» — корректный ответ RAG-системы^[1]. Правдоподобный ответ при нерелевантном retrieval — это дефект, а не «лучше, чем ничего».

[1] Lewis et al. 2020 — Retrieval-Augmented Generation (NeurIPS)

RAG — retrieval-augmented generation, поиск-дополненная генерация — это архитектура, в которой перед вызовом модели система сначала извлекает релевантные фрагменты из внешнего хранилища, кладёт их в контекст вместе с вопросом, и только затем модель генерирует ответ, опираясь на эти фрагменты[1]. Введём термин: retrieval — этап поиска и извлечения релевантных фрагментов; именно он отличает RAG от «положить документ в промпт вручную».

Ключевое: механизм retrieval в RAG — это ровно тот семантический поиск на эмбедингах, который мы построили в Лекции 2, и переобъяснять его не будем. Напомню одной фразой: текст превращается в вектор, близость векторов означает смысловую близость, поэтому фрагмент можно найти по смыслу вопроса, а не по совпадению слов. RAG надстраивает три шага. Первый — индексация, заранее, офлайн: корпус режется на фрагменты, каждый превращается в эмбединг и кладётся в векторное хранилище. Второй — retrieval, на запрос: вопрос превращается в эмбединг, по близости векторов достаются релевантные фрагменты. Третий — генерация с опорой: фрагменты кладутся в контекст вместе с вопросом, модель отвечает, опираясь на них, и в хорошей реализации — со ссылкой на источник.

Введём ещё один термин, несущий в разделе основную нагрузку. Grounding, опора на источник, — свойство ответа быть выведенным из конкретных извлечённых фрагментов, а не сочинённым моделью из головы, с возможностью показать, из какого фрагмента взят каждый факт. Хорошая RAG-система проектируется так, чтобы ответ был grounded, и чтобы на вопрос, на который в извлечённых фрагментах ответа нет, система говорила «не знаю» или «см. источник», а не сочиняла. Сформулируем как инженерный инвариант: «не знаю» — корректный ответ RAG-системы; правдоподобный ответ при нерелевантном

retrieval — дефект, а не «лучше, чем ничего». Различение «ответ с опорой и цитатой» против «правдоподобный текст» окажется центральным дальше и прямо объясняет дело Air Canada.

Источники:

[1] Lewis et al. 2020 — Retrieval-Augmented Generation (NeurIPS) — каноническая статья RAG: индексация → retrieval → генерация с опорой.
<https://arxiv.org/abs/2005.11401>

Когда RAG — правильный выбор.

RAG оправдан при сильном сигнале по признакам ниже — и отсутствии blockers со следующего слайда «когда НЕ RAG».

Большое / растущее

не влезает в окно целиком, или дорого класть весь корпус в каждый запрос

Меняется

документы, цены, регламенты обновляются чаще, чем выходят версии модели

Свежесть + провенанс

ответ опирается на проверяемый источник; можно показать, откуда факт

Приватная база

знания компании не входят в веса публичной модели

Признаки усиливают друг друга → RAG

Корп. база из тысяч регламентов, обновляется еженедельно, ответ с обязательной ссылкой на пункт: все признаки сошлись, ни один более простой механизм их совместно не закрывает → образцовый профиль RAG.



Один признак — повод присмотреться, но не строить автоматически: сверьтесь с блокерами на следующем слайде.

[1] IBM — RAG vs Fine-Tuning (вендор-нейтрально) · [2] U-NIAH — RAG win-rate выше у меньших моделей

15 / 40

RAG — правильный выбор, когда налицо сильный сигнал по одному или нескольким признакам[1] и при этом нет blockers с соседнего критерия «когда НЕ». Первый признак: знание большое или растущее — корпус не помещается в окно целиком, либо помещается, но класть его в каждый запрос дорого и медленно. Второй: знание меняется — документы, цены, регламенты обновляются чаще, чем выходят версии модели; RAG читает актуальное хранилище в момент запроса, и новый документ доступен сразу после индексации. Третий: нужны свежесть и провенанс — ответ должен опираться на проверяемый источник, и нужно показать, откуда взят факт; регулируемые домены почти всегда требуют провенанса. Четвёртый: база приватная — знания компании не входят в веса публичной модели.

Важна логика применения. Один явный признак — уже повод присмотреться к RAG, но не повод строить его автоматически: прежде чем решать, сверьте задачу с критериями «когда НЕ» — нет ли там блокера. Знание большое, но не меняется и помещается в окно — кандидат на длинный контекст, а не RAG. Знание меняется, но задача сводится к отдаче фиксированного значения — кандидат на детерминированный lookup. Признаки обычно усиливают друг друга: знание, одновременно большое, меняющееся, требующее провенанса и приватное, — это и есть профиль, под который RAG спроектирован, и чем больше признаков сходится, тем увереннее выбор. Формула короткая: сильный сигнал плюс отсутствие blockers.

Рабочий пример, к которому вернёмся как к разбору чек-листа в финале: корпоративная база из тысяч регламентов, обновляемых еженедельно, вопросы на естественном языке, обязательная ссылка на пункт-источник. Все четыре признака сошлись, и ни один более простой механизм их совместно не закрывает,

а проверка по критериям «когда НЕ» ничего не отменяет. Это образцовый профиль RAG. И практическое наблюдение: выигрыш RAG над прямым ответом особенно велик для меньших моделей — внешний retrieval компенсирует то, чего меньшей модели не хватает в весах, поэтому RAG нередко даёт нужное качество на более дешёвой модели.

Источники:

[1] IBM — RAG vs Fine-Tuning (вендор-нейтрально) — признаки-за-RAG:

большое/меняется/провенанс/приватное.

<https://www.ibm.com/think/topics/rag-vs-fine-tuning>

[2] U-NIAH — RAG win-rate выше у меньших моделей — выигрыш RAG над прямым ответом особенно велик для меньших моделей. <https://arxiv.org/abs/2503.00353>
[VFY-day-of]

Когда RAG — НЕ правильный выбор.

Знать, когда RAG не нужен, ценнее: это модная архитектура, её ставят туда, где она вредит.

1 Корпус влезает в окно



ориентир — менее ~200k токенов,
меняется редко
→ full-context + кэширование
префикса, не RAG-инфраструктура

2 Фиксированная политика / значение^[2]



тариф, цена, пункт регламента, правило
→ детерминированный lookup /
статическая страница

3 Данные доступны live через API / MCP



во внутреннем сервисе, базе, поиске другой системы
→ вызвать инструмент напрямую;
RAG-индекс поверх — лишний, более хрупкий и более устаревающий слой

RAG избыточен, если выполнен ЛЮБОЙ из трёх: (а) корпус влезает в окно и стабилен, (б) задача сводится к фиксированному значению, (в) знание уже доступно напрямую и живьём через инструмент. «Прямой вызов вернёт данные на момент запроса; RAG-индекс — на момент последней индексации.»

[1] Red Hat — RAG vs Fine-Tuning (когда не RAG) · [2] McCarthy Tétrault — Air Canada («генерация поверх фикс. политики»)

Знать, когда RAG не нужен, ценнее, чем знать, когда нужен, потому что RAG — модная архитектура, и её ставят туда, где она вредит. Три явных критерия «не RAG»[1].

Первый. Корпус помещается в окно — ориентир порядка менее двухсот тысяч токенов — и меняется не часто. Правильный ответ — full-context с кэшированием префикса, а не RAG-инфраструктура. Это проще, дешевле и не вносит риск «retrieval достал не то»: нет векторного хранилища, конвейера индексации, компонента retrieval, который может сломаться. RAG здесь куплен без необходимости.

Второй. Задача — отдать фиксированную политику или значение: тариф, цена, пункт регламента, правило. Если ответ детерминирован и заранее известен, правильная архитектура — детерминированный lookup или статическая страница, а не «retrieval плюс генерация поверх». Генерация поверх фрагмента всегда несёт риск, что модель перефразирует, обобщит или додумает — именно это произошло в деле Air Canada.

Третий критерий часто упускают. Данные уже доступны напрямую и живьём через API, MCP или поиск в другой системе. Если знание и так лежит во внутреннем сервисе с REST-интерфейсом или в базе, доступной через MCP, — строить отдельный RAG-индекс поверх него избыточно. RAG существует, чтобы дать доступ к знанию, до которого нет прямого пути; если путь уже есть, RAG добавляет лишний, более хрупкий и устаревающий слой. Разница стратегическая: прямой вызов инструмента возвращает данные на момент запроса, а RAG-индекс — на момент последней индексации. Признак прост: если на вопрос «а почему бы модели не спросить систему X напрямую через инструмент?» ответ —

«действительно, почему бы и нет», — RAG не нужен.

Сведём в правило: RAG избыточен, если корпус помещается в окно и стабилен, если задача сводится к фиксированному значению, или если знание уже доступно живьём через инструмент. Выполнен любой из трёх — сигнал остановиться. И даже когда ни один не сработал и RAG оправдан, это не гарантирует, что он будет работать хорошо на масштабе, — отдельный вопрос.

Источники:

[1] Red Hat — RAG vs Fine-Tuning (когда не RAG) — корпус влезает в окно → full-context+кэш; фикс. значение → lookup; live → API без индекса.
<https://www.redhat.com/en/topics/ai/rag-vs-fine-tuning>

[2] McCarthy Tétrault — Air Canada («генерация поверх фикс. политики») — фиксированная политика → детерминированный lookup, не retrieval+генерация.
<https://www.mccarthy.ca/en/insights/blogs/techlex/moffatt-v-air-canada-misrepresentation-ai-chatbot>

Провал RAG на масштабе.

«Вернул что-то» ≠ «вернул правильное»^[1,2]. У RAG нет сигнала «не нашёл» — он всегда отдаёт к ближайших, даже нерелевантных.



Legal-AI

«ближайшие» дела из другой юрисдикции / отменённый прецедент — модель опирается на них как на факт



Medical-RAG

смешал фрагменты разных пациентов — близки по симптомам, клинически объединять нельзя



Support-бот

работал на сотнях статей; после роста до тысяч качество тихо просело — никто не заметил

Air Canada — разбор архитектуры^[3]

Диагноз

сгенерированный правдоподобный текст поставлен в роль, требовавшую извлечённого проверенного факта — отказ grounding

Правильная альтернатива

фиксированная политика → lookup / страница; нужен диалог → RAG со strict grounding, обязательная цитата, явное «не знаю», проверка человеком

[1] Barnett et al. 2024 — Seven Failure Points (RAG) · [2] Kore.ai — Seven RAG Engineering Failure Points · [3] McCarthy Tétrault — Air Canada (отказ grounding) · Документированные классы провалов (Barnett et al. 2024; Air Canada — McCarthy Tétrault 2024). Кейсы — illustrative.

17 / 40

Главный урок раздела одной фразой: «система вернула что-то» не означает «система вернула правильное»[1,2]. RAG не имеет сигнала «я не нашёл подходящего» — по умолчанию он всегда возвращает к ближайших фрагментов, даже если они нерелевантны. Дальше модель честно делает свою работу: дали мусор — сочинит правдоподобный ответ поверх мусора.

Это не гипотетический риск, а известный паттерн инжиниринга RAG — семь точек отказа систематизированы у Barnett et al. (arXiv:2401.05856, 2024). Три класса. Legal-AI достаёт «ближайшие к» дел; по вектору близки дела со совпадающими словами, но юридически нерелевантные — другая юрисдикция, отменённый прецедент; модель встраивает их как опору. Векторная близость — это близость формулировок, не применимости. Medical-RAG: вопрос про одного пациента извлекает фрагменты, близкие по симптомам, но от других пациентов; модель объединяет то, что клинически объединять нельзя. Support-бот: работал на сотнях статей, после роста до тысяч качество незаметно просело — деградация на масштабе, потому что порога «достаточно ли близко» нет. Альтернатива везде — не «убрать RAG», а сделать его наблюдаемой системой: эталонный проверочный набор и алерты, нарезка по смысловым границам, гибридный поиск с переранжированием.

Теперь вернёмся к Air Canada[3] (дело *Moffatt v. Air Canada*, BC CRT, решение 14.02.2024) — с инструментами раздела. Бот сообщил про возврат разницы и сослался на страницу с реальной политикой; политика на той же странице такого возврата не допускала. Источник истины существовал, был доступен — но ответ был не выведен из него, а сгенерирован как правдоподобный текст. Это образцовый отказ grounding: сгенерированный текст в роли, требовавшей извлечённого факта. Правильная архитектура: для фиксированной политики —

детерминированная страница или lookup; если нужен диалог — RAG со строгой опорой, обязательной цитатой, явным «не знаю» и человеческой проверкой. Air Canada — не «ИИ галлюцинирует», это решение поставить генеративную архитектуру на детерминированную задачу. Короткий вопрос вам: RAG вернул ответ — как узнать, что он правильный? Только измерением; на глаз правдоподобный от верного не отличить.

Источники:

[1] Barnett et al. 2024 — Seven Failure Points (RAG) — «вернул что-то ≠ вернул правильное»: 7 точек отказа RAG-инжиниринга. <https://arxiv.org/abs/2401.05856>

[2] Kore.ai — Seven RAG Engineering Failure Points — legal-AI / medical-RAG / support-бот — деградация на масштабе без observability.
<https://www.kore.ai/blog/seven-rag-engineering-failure-points>

[3] McCarthy Tétrault — Air Canada (отказ grounding) — сгенерированный текст в роли, требовавшей извлечённого проверенного факта.
<https://www.mccarthy.ca/en/insights/blogs/techlex/moffatt-v-air-canada-misrepresentation-ai-chatbot>

РАЗДЕЛ 3

Fine-tune vs промпт vs RAG

Проблему знания мы решили через RAG. А если проблема не в знании, а в поведении модели — её тоном, форматом, политикой?



Раздел 0
Открытие

Раздел 1
Промпт

Раздел 2
RAG

Раздел 3
Fine-tune

Раздел 4
Агенты

Раздел 5
Фреймворк

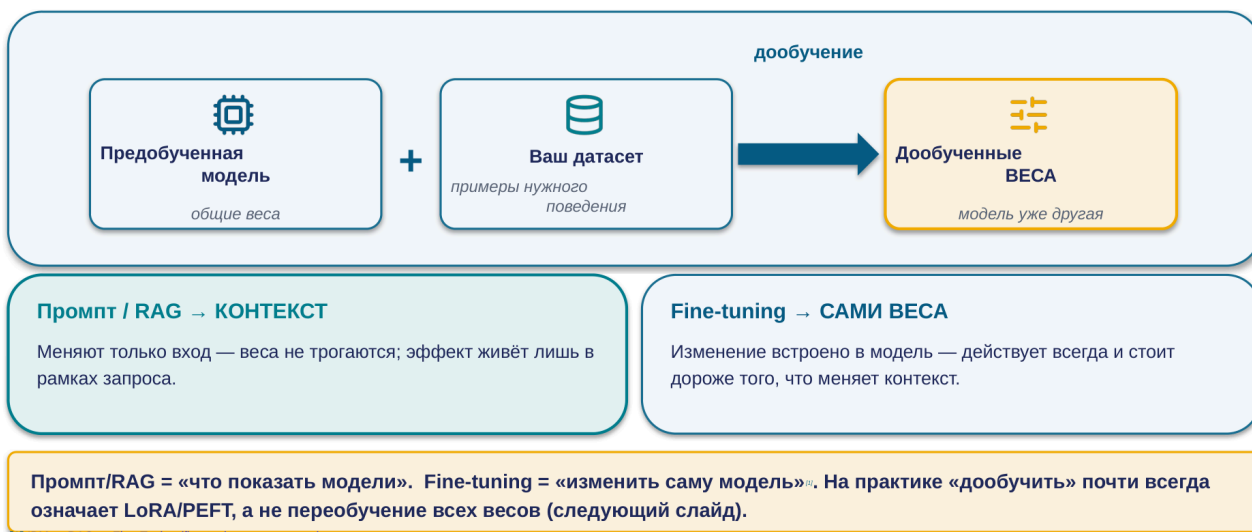
18 / 40

В прошлом разделе мы разобрали RAG и закрыли им один конкретный класс проблем — проблему знания. Когда модели не хватает фактов: они приватные, или меняются быстрее, чем выходят новые версии модели, или нужен провенанс — мы не переучиваем модель, мы подкладываем нужные документы в контекст. Мы также честно разобрали, когда RAG не нужен и как он тихо ломается на масштабе, и замкнули раздел обратно на дело Air Canada.

Но не всякая проблема — это проблема знания. Бывает, что фактов модели хватает, а не устраивает само поведение: она отвечает не в том тоне, не в том формате, не следует доменной политике, или вам нужна маленькая дешёвая модель, повторяющая поведение большой. Это другой класс задачи, и RAG его не решает — сколько документов ни подкладывай, тон от этого не изменится. Здесь на сцену выходит третий инструмент — fine-tuning, дообучение. И сразу важная оговорка по структуре раздела: следующий слайд — это не критика и не «когда не надо», а определение. Прежде чем разбирать, где fine-tuning сузился, где он проваливается через катастрофическое забывание и по каким критериям выбирать между ним, RAG и промптом, нужно одинаково понимать, что это вообще такое и чем оно принципиально отличается от двух уже знакомых инструментов. С определения и начнём.

Что такое fine-tuning.

Fine-tuning (дообучение) — продолжение обучения уже готовой модели на ваших данных. В Л1 — тип использования; здесь — архитектурный выбор, одна из ступеней лестницы.



[1] IBM — RAG vs Fine-Tuning (fine-tuning меняет веса)

19 / 40

Прежде чем что-либо говорить про то, где fine-tuning сузился, где он опасен и по каким критериям его выбирать, нужно одинаково понимать, что это такое. Fine-tuning, по-русски дообучение, — это продолжение обучения уже готовой, предобученной модели[1] на ваших собственных данных. В Лекции 1 дообучение упоминалось мельком как один из типов использования ИИ; здесь оно появляется в своей настоящей роли — как архитектурный выбор, одна из ступеней той же лестницы сложности.

Механика на схеме: берётся предобученная модель с её общими весами, берётся ваш датасет — набор примеров того поведения, которое вы хотите получить, — и процесс дообучения сдвигает веса модели в сторону этих примеров. На выходе — уже другая модель: те же архитектура и размер, но другие числа внутри. Это и есть ключевое отличие, ради которого нужен отдельный слайд-определение. Промпт и RAG не трогают веса вообще: они меняют только контекст, то есть то, что вы подаёте модели на вход прямо сейчас. Эффект промпта или подложенного RAG-фрагмента живёт ровно в пределах одного запроса и исчезает со следующим. Fine-tuning меняет саму модель: изменение встроено в веса, действует на каждом запросе по умолчанию, и именно поэтому стоит дороже и откатывается тяжелее всего, что меняет лишь контекст.

Важная практическая оговорка сразу: на практике «дообучить модель» почти всегда означает не переобучение всех весов, а параметр-эффективное дообучение, которое обычно называют аббревиатурой PEFT и чаще всего реализуют методом LoRA. Полное дообучение всех весов в 2026 году — редкость, и на следующих слайдах мы разберём, почему это так. Пока запомните различие как формулу: промпт и RAG — это «что показать модели», fine-tuning — это «изменить саму модель». На следующих слайдах мы будем опираться на это

определение, разбирая, почему в 2026 году fine-tuning сузился до поведения, а не знания, и где он превращается из инструмента в источник проблем.

Источники:

[1] IBM — RAG vs Fine-Tuning (fine-tuning меняет веса) — промпт/RAG меняют контекст; fine-tuning меняет сами веса модели.

<https://www.ibm.com/think/topics/rag-vs-fine-tuning>

PEFT вместо full fine-tuning.

PEFT — базовые веса замораживаются, обучается лишь небольшой набор адаптеров. LoRA — низкоранговые матрицы-адаптеры; QLoRA — то же поверх квантованной модели^[3].



[1] Hu et al. 2021 — LoRA · [2] Dettmers et al. 2023 — QLoRA · [3] HF PEFT — Beyond LoRA (LoRA 98,4% из 20 834 карточек) · HF PEFT team, «Beyond LoRA?», июнь 2026. Полный спектр методов (SFT / DPO / RFT) — в главе.

20 / 40

Слово «дообучить модель» у многих ассоциируется с обновлением всех весов — это full fine-tuning. В 2026 это почти никогда не то, что нужно. PEFT, параметр-эффективное дообучение, — семейство методов, при которых базовые веса замораживаются, а обучается лишь небольшой набор дополнительных параметров, адаптеров[1]. Наиболее распространённый метод — LoRA: в выбранные слои добавляются маленькие низкоранговые матрицы-адаптеры, и обучаются только они. QLoRA — та же идея поверх квантованной базовой модели, что радикально снижает требования к памяти.

Почему PEFT почти всегда предпочтительнее full fine-tuning — три причины, и третья самая важная. Первая: дешевле и быстрее — обучаются миллионы параметров-адаптеров вместо миллиардов весов; QLoRA позволяет дообучать большие модели на одном GPU. Вторая: модульность — адаптеры маленькие, мегабайты против гигабайтов, и можно держать несколько адаптеров под разные задачи на одной замороженной базе. Третья, главный архитектурный аргумент про надёжность: базовые веса заморожены, физически не переписываются под новый сигнал, поэтому то, что модель умела до дообучения, в основном сохраняется — обучение живёт в маленьких добавочных адаптерах. Это прямо снижает риск катастрофического забывания, к которому мы сейчас перейдём.

Насколько LoRA распространён — не оценка на глаз, а измеримый факт. По данным команды PEFT Hugging Face, среди 20 834 карточек моделей с тегом PEFT 98[3].4 процента используют именно LoRA. Оговорка обязательна: это доля среди уже помеченных тегом PEFT, а не среди всего fine-tuning — полное дообучение помечается тегами не так аккуратно и в выборке может быть не видно. Тем не менее среди выбравших параметр-эффективный путь LoRA — практически безальтернативный дефолт. И граница: и full FT, и PEFT — это изменение весов, в

отличие от RAG, промпта и контекста, которые веса не трогают. Это водораздел между «разместить знание» — RAG и контекст — и «закодировать поведение» — PEFT. В нарративе достаточно: full FT — почти никогда, PEFT и LoRA — рабочая лошадка 2026 года.

Источники:

[1] Hu et al. 2021 — LoRA — базовые веса замораживаются, обучаются низкоранговые адаптеры. <https://arxiv.org/abs/2106.09685>

[2] Dettmers et al. 2023 — QLoRA — LoRA поверх квантованной модели → дообучение на одном GPU. <https://arxiv.org/abs/2305.14314>

[3] HF PEFT — Beyond LoRA (LoRA 98,4% из 20 834 карточек) — 98,4% моделей с тегом PEFT используют LoRA; оговорка: доля среди помеченных PEFT. <https://huggingface.co/blog/peft-beyond-lora> [VFY-day-of]

Fine-tuning не умер^[1] — он сузился до поведения.

**Ушло из fine-tuning (2026)**

знание, факты, то, что меняется → RAG / длинный контекст. Защитить факты в весах: нельзя процитировать, обновить точно, удалить один.

**Осталось за fine-tuning**

поведение · стиль · формат вывода · следование политике. Дистилляция — отдельная техника, часто применяемая в связке с дообучением».

Если задача требует...	→ правильный инструмент	...и НЕ этот, потому что
знание меняется / нужны свежесть, провенанс	RAG (или длинный контекст для малого корпуса)	не fine-tuning: устареет, дорого переобучать, риск forgetting
стабильное поведение / тон / формат / политика	fine-tuning (PEFT)	не RAG: подаёт знание, но не меняет манеру модели
снизить стоимость / латентность на узкой задаче	fine-tune teacher + дистилляция student	две отдельные техники в связке, НЕ «дистилляция = вид fine-tuning»
детерминированный, верифицируемый ответ	обычный код, без ИИ	ни RAG, ни FT: ИИ добавит недетерминизм без выигрыша

Реальный вопрос 2026 — не «RAG или fine-tuning», а «что здесь знание (→ RAG), что поведение (→ PEFT), что детерминировано (→ код)». Гибрид — норма там, где есть И проблема знания, И проблема поведения.

[1] BigData Boutique — Fine-Tuning When RAG Isn't Enough · [2] Hinton, Vinyals, Dean 2015 — Distilling the Knowledge · [3] PEFT survey — таксономия методов настройки

21 / 40

На предыдущем слайде мы зафиксировали определение: fine-tuning меняет сами веса, тогда как промпт и RAG трогают только контекст. Опираясь на это, разберём область его применения в 2026 году. Среди инженеров ходит формулировка «в 2026 fine-tuning умер — всё решает RAG и длинный контекст». Это неточно. Fine-tuning не умер — он сузился[1] и перестал быть настройкой по умолчанию. Сузился именно потому, что меняет веса: знание — факты, документы, всё, что меняется, — плохо ложится в веса и ушло в RAG и длинный контекст, где его можно процитировать и обновить. За fine-tuning осталась узкая, но важная область — стабильное поведение, стиль, формат вывода, следование политике, а не знания модели.

Отдельно и аккуратно про дистилляц[2,3]ию, потому что здесь частая путаница. Дистилляция — самостоятельная техника переноса знания и сжатия модели (Hinton, Vinyals, Dean, 2015): student-модель учится имитировать выходы teacher-модели через отдельную функцию потерь. Таксономически это не разновидность fine-tuning — обзоры методов разносят их по разным категориям. На практике две техники часто комбинируются: сначала teacher-модель дообучают под нужное поведение, а затем из неё дистиллируют компактную student-модель. Держите это как «fine-tune teacher, потом дистилляция student — две отдельные техники в связке», а не «дистилляция как вид fine-tuning».

Отсюда критерий что-куда. Знание меняется или нужен провенанс — RAG или длинный контекст, не fine-tuning: знание устареет, переобучение дорого, есть риск забывания. Нужно стабильное поведение, тон, формат, политика — это fine-tuning в форме PEFT, не RAG. Нужно снизить стоимость на узкой задаче — связка fine-tune teacher плюс дистилляция student. Ответ детерминирован и верифицируем — обычный код без ИИ. Иначе говоря, fine-tuning 2026 — узкий

инструмент под поведение, формат, политику и эффективность, а не способ научить модель домену; «научить домену» в смысле фактов — это RAG, в смысле манеры — PEFT, и почти никогда full fine-tuning.

Источники:

[1] BigData Boutique — Fine-Tuning When RAG Isn't Enough — fine-tuning сузился до поведения/стиля/формата/политики; знание → RAG.

<https://bigdataboutique.com/blog/fine-tuning-llms-when-rag-isnt-enough> [VFY-day-of]

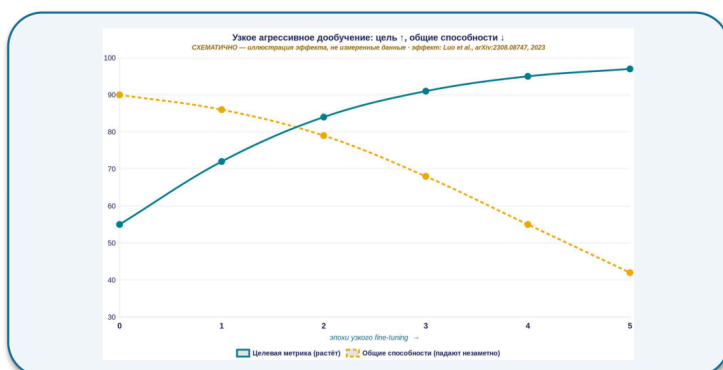
[2] Hinton, Vinyals, Dean 2015 — Distilling the Knowledge — дистилляция — самостоятельная техника, таксономически НЕ вид fine-tuning.

<https://arxiv.org/abs/1503.02531>

[3] PEFT survey — таксономия методов настройки — обзоры разносят дистилляцию и fine-tuning по разным категориям. <https://arxiv.org/abs/2403.14608>

Провал: catastrophic forgetting.

Catastrophic forgetting (катастрофическое забывание) — деградация общих способностей модели в результате узкого агрессивного дообучения.



тяжелее с ростом масштаба модели — у крупной выше исходный уровень, ей «больше падать»

Нет eval-петли на общих задачах + нет версий датасета/весов → не увидишь поломку до прода + не сможешь откатиться → это не «риск», это критерий «НЕ делай fine-tuning»

Правильно: PEFT (замороженные веса — ниже риск); для меняющегося знания — RAG, не FT вовсе.

[1] Luo et al. 2023 — Catastrophic Forgetting in LLM Continual FT · Эмпирически наблюдается при continual fine-tuning (Luo et al. 2023). Механизмы — «исследования показывают» (preprint).

22 / 40

Введём термин через документированный провал. Catastrophic forgetting, катастрофическое забывание, — деградация общих способностей модели[1] в результате узкого агрессивного дообучения: обучая модель быть очень хорошей в одной узкой задаче, вы можете сломать то, что она умела до этого.

Как это выглядит как кейс. Команда дообучает модель на узком датасете — например, классификация тикетов в их формат. На целевой метрике рост: модель отлично классифицирует. Параллельно, незаметно, проседают общие способности: рассуждение на нецелевых задачах, следование сложным инструкциям. На графике это две линии: сплошная, идущая вверх, — целевая метрика (растёт), а пунктирная, идущая вниз, — общие способности (падают незаметно). Если команда мерила только целевую метрику, деградация не видна, пока модель не начнут использовать на чём-то, кроме узкой задачи. И тогда «почему она вдруг хуже стала рассуждать?» оказывается следствием дообучения месячной давности, к которому уже нет ни старых весов, ни версии датасета. Механически: обучение под узкую задачу двигает веса в её сторону, и параметры, кодировавшие общие способности, при агрессивном full-FT переписываются. Это эмпирически наблюдается при continual fine-tuning (Luo et al., arXiv:2308.08747, 2023), причём с контринтуитивной особенностью: с ростом масштаба модели тяжесть забывания, как правило, возрастает. «Возьмём модель побольше, чтобы надёжнее» здесь работает наоборот.

Почему это попадает в раздел про выбор архитектуры, а не в курьёзы. Сам по себе forgetting — управляемый риск. Катастрофическим его делает отсутствие инженерной дисциплины: нет петли оценки, которая на репрезентативном наборе общих задач показала бы деградацию; нет версионирования датасета и весов для отката. Без eval-петли вы не заметите, что сломали общие способности,

пока это не всплывёт в проде; без версионирования не вернётесь к работавшей версии. Правило: если нет eval-петли и версионирования датасета — не делайте fine-tuning. Это не «риск», это критерий: не будет ни сигнала о поломке, ни кнопки отката. Правильные альтернативы: PEFT, где замороженные веса дают ниже риск; а для меняющегося знания — RAG, а не fine-tuning вовсе.

Источники:

[1] Luo et al. 2023 — Catastrophic Forgetting in LLM Continual FT — узкий агрессивный FT ломает общие способности; тяжелее с ростом масштаба модели.
<https://arxiv.org/abs/2308.08747>

РАЗДЕЛ 4

Агенты

От собеседника в окне чата — к компоненту продакшен-системы: цикл, экипировка, память, доступ к инструментам — и где всё это ломается.



Раздел 0
Открытие

Раздел 1
Промпт

Раздел 2
RAG

Раздел 3
Fine-tune

Раздел 4
Агенты

Раздел 5
Фреймворк

23 / 40

Это самый большой раздел лекции, и он про то, как AI перестаёт быть собеседником в окне чата и становится компонентом системы. До сих пор один вызов и RAG мы обсуждали как способ получить текст; здесь модель получает возможность возвращать машинно-обрабатываемые данные, дотягиваться до внешних систем и работать в цикле. Мы разберём механику API-слоя и стандарт подключения инструментов, затем сам цикл агента и различие рабочего потока и агента, потом — из чего собран реальный агент-помощник, где живёт его память и как всё это ломается. Здесь же по-настоящему встаёт сквозная тема безопасности: как только модель получает инструменты, появляется вопрос, которого не было у одного вызова, — кто видит данные на каждом шаге и что происходит, когда недоверенный контент попадает в контекст как команда. Это раздел, где цена неправильного выбора архитектуры измеряется не только деньгами, но и утечками.

Модель становится компонентом системы: API + MCP.



Structured output

выход строго по схеме (JSON), не текст для парсинга

встраиваемая



Function calling

модель формулирует «вызовы X»; исполняет ваш код, не модель

активная



Prompt caching

не пересчитывать неизменный префикс при каждом запросе

экономичная



MCP — «USB-C для инструментов LLM»

N×M несовместимых интеграций → **N+M**^[1]

инструмент один раз = MCP-сервер; модель один раз = MCP-клиент



Anthropic
11/2024



OpenAI
03/2025



Google
04/2025

Стандартизация подключения ≠ безопасность подключаемого — и усугубляет проблему доверия.

- код в вашем окружении / доступ к данным
- описание попадает в контекст — носитель prompt injection

Ни один механизм не делает модель надёжнее — правило лестницы не отменяется. Удобство подключения — не аргумент за подключение.

[1] Anthropic — MCP donation / Agentic AI Foundation (N×M → N+M) · Актуальные цифры экономики и масштаб экосистемы MCP — в главе методички.

24 / 40

Чтобы AI стал частью системы, нужны три механизма API-слоя. Первый — structured output, структурированный вывод: режим, в котором модель обязана вернуть ответ строго по заданной схеме, например JSON, а не свободный текст. Технически схема компилируется в ограничение на сэмплинг, поэтому модель не может выдать структурно невалидный ответ; выход превращается из текста, который надо парсить регулярками, в гарантированно валидные по форме данные. Это делает модель встраиваемой. Тонкость: гарантируется валидность формы, но не содержания — поле может быть фактически неверным. Второй — function calling: модель вместо ответа текстом может вернуть запрос «вызови инструмент X с аргументами Y», а исполняет вызов ваш код, не модель. Модель лишь формулирует намерение — это делает её активной и будет ключевым для безопасности. Третий — prompt caching: переиспользование уже вычисленного состояния неизменного префикса, чтобы не пересчитывать его каждый раз; кэшируется состояние, а не текст ответа. Это делает модель экономически жизнеспособной при объёме и делает full-context для малого стабильного корпуса реальным конкурентом RAG. Ни один из трёх не делает модель надёжнее или аудируемее — они расширяют возможности, не меняя недетерминированной природы, и правило лестницы не отменяют.

Остаётся вопрос, как инструмент к модели подключается. До конца 2024 года каждая интеграция решала это по-своему. MCP, Model Context Protocol, — открытый стандарт единого способа подключать^[1] инструменты к модели; метафора — «USB-C для инструментов». Суть в арифметике: при N моделях и M инструментах наивно нужно N умножить на M интеграций, а MCP сводит это к N плюс M. Но здесь критический поворот: стандартизация подключения не означает безопасность подключаемого и даже усугубляет проблему доверия. Каждый

подключённый инструмент — это код в вашем окружении и канал, через который в контекст попадает недоверенный текст. «Подключить за минуту» и «доверять подключённому» — разные вопросы; MCP решил первый и сделал второй острее. Удобство подключения — не аргумент за подключение.

Источники:

[1] Anthropic — MCP donation / Agentic AI Foundation ($N \times M \rightarrow N + M$) — MCP стандартизует подключение; удобство подключения $\neq$ безопасность подключаемого.

<https://www.anthropic.com/news/donating-the-model-context-protocol-and-establishing-of-the-agentic-ai-foundation> [VFY-day-of]

Цикл агента: plan → act → check → iterate.

Агент — архитектура, где модель не делает один проход, а работает в цикле, сама определяя последовательность шагов.



[1] Yao et al. 2022 — ReAct (plan → act → check → iterate) · [2] Anthropic — Reasoning Models faithfulness (check ≠ самооценка) · Паттерны цикла (ReAct, Reflexion, Plan-and-Execute) — в главе. Проектировать агента = проектировать защиту на каждом из 4 шагов.

25 / 40

Теперь есть всё для следующей ступени. Один вызов умеет рассуждать; RAG достаёт знание; function calling дотягивается до внешних систем; MCP стандартизует подключение. Агент — архитектура, в которой модель не делает один проход, а работает в цикле[1]: планирует шаг, действует — вызывает инструмент, наблюдает результат, проверяет, достигнута ли цель, и итерирует, сама динамически определяя последовательность. Каноническая формулировка — планируй, действуй, проверяй, повторяй; это паттерн ReAct, чередование рассуждения и действий, а Reflexion добавляет явную самооценку после неудачи. Это «другой AI» по сравнению с одним вызовом: там модель отвечала один раз, здесь — управляет процессом из многих шагов.

Разберём цикл по шагам, потому что каждый шаг — место конкретного режима отказа. Plan: модель формулирует шаг исходя из цели и истории; режим отказа — близорукий или заикленный план, модель не видит накопленную стоимость и время. Act: вызывается инструмент; режим отказа — инструмент падает или возвращает ошибку, и если плана на это нет, начинается петля. Check: оценивается результат; режим отказа — проверки нет или она сводится к самооценке модели. Iterate: цикл повторяется; режим отказа — нет внешнего лимита на итерации, стоимость или время. «Агент» — не одна абстракция, а четыре шага, каждый со своим способом сломаться; проектировать агента значит проектировать защиту на каждом из четырёх.

Подчеркну элемент, несущий основную сквозную нагрузку курса, — шаг check[2]. И здесь возвращается урок про верность рассуждения ребром: проверка не должна быть «модель сама себя спросила и сама себе ответила, что всё хорошо». Самооценка подвержена той же unfaithfulness, что мы видели на слайде про предел chain-of-thought — это иллюзия контроля: объяснение «я проверил»

порождается тем же сэмплингом, что и сам ответ. Надёжная проверка — валидация против внешнего критерия: схема, тест, проверка инвариантов, сверка с источником истины, а в значимых решениях — человек-валидатор. Без внешнего check цикл агента — петля, которая уверенно идёт не туда. Это прямой мост к провалам агентов.

Источники:

[1] Yao et al. 2022 — ReAct (plan→act→check→iterate) — чередование рассуждения и действий; каждый шаг — место отказа. <https://arxiv.org/abs/2210.03629>

[2] Anthropic — Reasoning Models faithfulness (check ≠ самооценка) — шаг check — валидация против внешнего критерия, не самооценка модели.

<https://www.anthropic.com/research/reasoning-models-dont-say-think> [VFY-day-of]

Workflow vs Agent.

Предсказуемая задача → workflow; непредсказуемая И ценность оправдывает кратный рост → агент.



Workflow (рабочий поток)

LLM и инструменты по предопределённым в коде путям

- последовательность шагов известна заранее
- предсказуемо, аудируемо
- большинство надёжных продакшн-систем — это workflow



Агент

LLM динамически определяет собственный процесс

- последовательность заранее не зафиксирована
- кратно больше токенов, чем чат
- ниже аудируемость, выше риск петель

Могу ли я заранее, до запуска, выписать последовательность шагов? да (даже с ветвлениями) → workflow · принципиально нет И ценность оправдывает кратные стоимость/риск → агент

Вложенность — норма: агент code review вызывает workflow «линтер → тесты → формат» как один шаг; workflow обработки заявок делегирует мини-агенту разбор свободной жалобы. Динамика — только там, где нужна.

Найди простейшее. «Лень формализовать» не делает задачу непредсказуемой.

[1] Anthropic — Building Effective Agents (workflow vs agent) · [2] Cognition — Don't Build Multi-Agents

26 / 40

Перед тем как разбирать, как агенты ломаются, нужно ввести различие, для основной учебной цели лекции — одно из самых важных. Anthropic в «Building Effective Agents» разделяет два понятия[1], которые в обиходе часто смешивают. Workflow, рабочий поток или сценарий, — система, где модель и инструменты оркестрируются по предопределённым в коде путям; последовательность шагов известна заранее и зашита, модель вызывается на конкретных шагах, но не решает, какие шаги и в каком порядке делать. Агент — система, где модель динамически определяет собственный процесс; последовательность заранее не зафиксирована.

Различение операциональное и даёт прямой критерий выбора. Задача предсказуемая, последовательность известна — workflow. Непредсказуемая, последовательность зависит от промежуточных результатов, и ценность оправдывает кратный рост стоимости — агент. Anthropic формулирует размен прямо: агентные системы меняют латентность и стоимость на качество. Переведём критерий в один практический вопрос: могу ли я заранее, до запуска, выписать последовательность шагов? Если да, даже с известными ветвлениями, — это workflow. Если последовательность принципиально зависит от промежуточных результатов, и ценность оправдывает рост риска, — агент. «Лень формализовать» не делает задачу непредсказуемой. Это, возможно, самая дорогая ошибка отрасли: строить динамического агента там, где хватило бы workflow.

Важная оговорка: граница проходит не только по задаче в целом, но и по тому, что происходит внутри конкретного шага, — workflow и агент вкладываются друг в друга, и это норма. Агент на шаге действия может вызвать целый workflow как один инструмент: агент code review сам решает, что проверять, но шаг «линтер,

тесты, форматирование» вызывает как единый детерминированный подпроцесс. И наоборот: workflow обработки заявок ведёт маршрут по фиксированным правилам, но шаг «разобраться в свободной жалобе» делегирует мини-агенту. Это не третья архитектура, а признание того, что вопрос «workflow или агент» — про конкретное место в системе: динамика добавляется только там, где нужна. Полный дебат о мульти-агент[2]ах остаётся в главе; в нарративе достаточно: мульти-агент по умолчанию — не апгрейд.

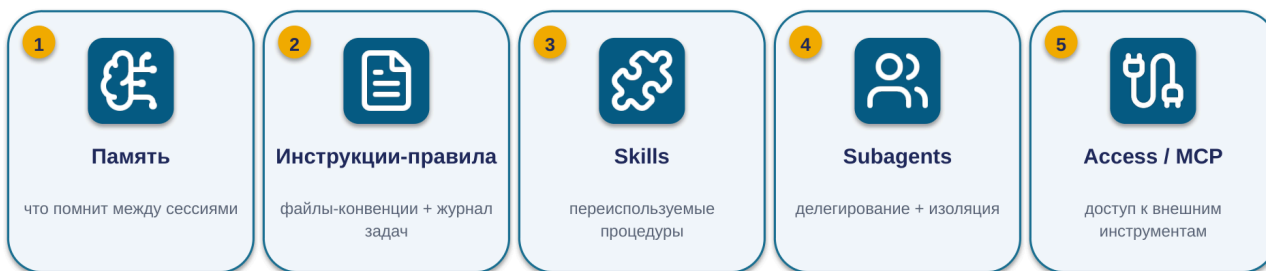
Источники:

[1] Anthropic — Building Effective Agents (workflow vs agent) — workflow = предопределённые пути; агент = динамический процесс; размен латентность/стоимость↔качество.
<https://www.anthropic.com/research/building-effective-agents>

[2] Cognition — Don't Build Multi-Agents — мульти-агент по умолчанию не апгрейд; хрупкость параллельных субагентов. <https://cognition.ai/blog/dont-build-multi-agents>

Каждый слот экипировки агента — компромисс, не апгрейд.

Реальный агент-помощник (Claude Code, Cursor, Aider) — это цикл *plan* → *act* → *check* → *iterate* ПЛЮС оснастка. Пять типовых слотов:



Как не поднимаешься по лестнице архитектур без требования задачи — не добавляй агенту память, subagents или MCP «на всякий случай». Каждый слот несёт свою цену: операционную сложность, новую поверхность отказа, новую границу доверия — и должен отвечать на конкретный триггер (Раздел 5).

[1] agent-harness-registry — карта слотов экипировки агента

27 / 40

До сих пор мы говорили про агента как про абстрактный цикл: планируй, действуй, проверяй, повторяй. Но реальный агент-помощник, с которым инженер работает каждый день, — это не голый цикл, а цикл плюс оснастка[1]. Набор дополнительных механизмов, которые определяют, что агент помнит, чему следует, какие процедуры умеет переиспользовать, кому может делегировать работу и к каким внешним системам имеет доступ. Дальше в этом разделе мы будем не раз опираться на данные agent-harness-registry — это независимый публичный реестр, который тестирует экипировку агентов (память, skills, subagents, MCP) через live-eval бенчмарки на реальных задачах, а не через вендорские самоотчёты о возможностях. По его данным можно выделить пять типовых слотов этой оснастки. Первый — память: что агент помнит между сессиями. Второй — инструкции-правила: файлы-конвенции проекта и журнал прогресса. Третий — skills, переиспользуемые процедуры под повторяющиеся задачи. Четвёртый — subagents, делегированные под-агенты с собственным контекстным окном. Пятый — доступ к внешним инструментам через MCP.

Прежде чем разбирать каждый слот по отдельности, зафиксируем тезис, который рифмуется с центральным правилом всей лекции и с лестницей архитектурной сложности. Каждый слот экипировки — это компромисс между ценой, сложностью и отказоустойчивостью, а не апгрейд по умолчанию. Ровно как не стоит подниматься по лестнице архитектур без требования задачи, не стоит и добавлять агенту память, subagents или новый доступ на всякий случай: каждый добавленный слот несёт свою цену — операционную сложность, новую поверхность отказа, новую границу доверия — и должен отвечать на конкретный, а не абстрактный триггер. Дальше мы пройдем вдоль этой карты слот за слотом и на каждом проверим этот тезис.

Источники:

[1] agent-harness-registry — карта слотов экипировки агента — источник не подтверждён canonical-URL 2026-08-30; Claude Code/Cursor/Aider — вендор-сайты, verify day-of. [VFY: не подтверждён canonical-URL, подавать как данные независимого реестра live-eval, не как первоисточник]

Память агента — тот же вопрос масштаба, что RAG.

Память — что агент помнит *МЕЖДУ* сессиями (в отличие от контекста одного разговора). Спектр — от плоского файла до graph-баз.



Плоский файл

агент дописывает факты в текстовый лог, при запуске читает целиком. Работает, пока лог мал и стабилен — прямая параллель критерию RAG «корпус помещается в окно».



Векторная / graph-база

- mem0 — cross-session память пользователя
- Cognee — memory на графе знаний
- Graphiti / Zep — temporal graph: когда факт верен, когда устарел

Тот же вопрос масштаба знания, что решает RAG для корпуса документов, здесь встаёт для памяти самого агента. Не ставьте graph-базу агенту с короткими несвязанными сессиями — это тот же технический долг без требования, что RAG для десяти статей.

[1] agent-harness-registry (live-eval) — спектр памяти агента · Источник: публичный реестр agent-harness-registry (workain lab, live-eval).

28 / 40

Первый слот экипировки — память: то, что агент помнит между сессиями, в отличие от контекста одного разговора, который живёт, только пока открыт диалог. Простейшая форма памяти — плоский файл[1]: агент дописывает в текстовый лог факты, выводы, историю решений, и при следующем запуске читает этот файл целиком. Это работает, пока лог небольшой и не растёт бесконтрольно. На другом конце спектра — специализированные векторные или графовые базы памяти. Система mem0 собирает факты о пользователе и его предпочтениях и переиспользует их в будущих сессиях. Cognee строит память на графе знаний, где факты связаны через онтологию, и retrieval учитывает не только текстовую близость, но и связи между сущностями. Graphiti и Zep идут дальше — это temporal knowledge graphs, которые явно отслеживают время действия факта и его происхождение: система знает, когда факт стал верным, когда устарел и откуда взят.

Здесь прямо повторяется вопрос, который мы уже разбирали для RAG. Тот же вопрос масштаба знания, что решает RAG для корпуса документов, встаёт для памяти самого агента. Плоский файл работает, пока история мала и стабильна, — это прямая параллель критерию «корпус помещается в окно». Структурированная база нужна, когда история большая, растущая или требует извлечения по фактам — кто что сказал, когда, актуально ли ещё, — а не просто «дай последние сообщения». Критерий один и тот же, применён к другому объекту. И симметрично работает правило лестницы: не ставьте графовую базу памяти агенту, у которого короткие несвязанные сессии и нечего структурированно вспоминать, — это тот же технический долг без требования, что RAG для десяти статей.

Источники:

[1] agent-harness-registry (live-eval) — спектр памяти агента — источник не подтверждён canonical-URL 2026-08-30; параллель с критерием масштаба RAG.
[VFY: не подтверждён canonical-URL, подавать как данные независимого реестра live-eval, не как первоисточник]

«Агент, который помнит» — не всегда лучше.

Наличие памяти интуитивно кажется чистым улучшением. Независимый live-eval показывает: иногда — драматически нет».



[1] agent-harness-registry (live-eval) — Letta Tier D / Memory Tool 17%-хаос · agent-harness-registry (workain lab, live-eval 2026-07-05). «Работает хорошо» ≠ «работает всегда» — тот же урок, что RAG на масштабе u forgetting.

29 / 40

Наличие памяти интуитивно кажется чистым улучшением — агент, который помнит, должен быть полезнее агента, начинающего каждый раз с нуля. Данные agent-harness-registry, независимого реестра[1], тестирующего системы на реальных задачах, показывают, что это не всегда так, а иногда драматически не так. На слайде системы помечены буквой Tier — это рейтинговая категория реестра от A (лучшие результаты) до D (худшие) по сумме бенчмарков live-eval; чем ближе к A, тем стабильнее система показывала себя на полном наборе тестов. Первый кейс — система Letta, Tier D. По данным реестра она проигрывает и голой модели без всякой памяти, и тривиальному плоскому файлу на всех полностью протестированных бенчмарках: на persistbench голая модель дала 1.000, плоский файл 0.833, а Letta лишь 0.750, при этом на порядок медленнее. Выявлены три механизма провала: капитуляция под давлением, когда при повторном вопросе система меняет уже данный верный ответ на неуверенный; многословие, когда длинный приукрашенный ответ прячет короткий правильный факт; и факт, замеченный, но не закоммиченный в память. Оговорка по свежести обязательна: тестировалась версия примерно восемнадцатимесячной давности из-за ограничения установщика, а не из-за срезания угла, поэтому это оценка конкретной устаревшей версии.

Второй кейс важнее, потому что это лучшая протестированная система. Anthropic Memory Tool в целом сильна, но и у неё в четырёх задачах из двадцати четырёх, то есть в 17 процентах, зафиксирована потеря информации: явный отказ записать факт, сочтённый эфемерным; отказ как «вне рамок»; и тихая суммаризация, когда конкретные факты незаметно поглощаются общими категориями. Самая тревожная деталь — невоспроизводимость: одна и та же беседа, поданная дважды, дала разное поведение. Это не баг, а

задокументированный дизайн: модель просят применять суждение, что достойно записи, и это суждение производит реальные, иногда противоречивые провалы. Вывод один: даже за лучшую систему памяти стоит измеримый хвост потерь. «Работает хорошо» и «работает всегда» — разные утверждения, и это тот же урок, что RAG-провал на масштабе и катастрофическое забывание.

Источники:

[1] agent-harness-registry (live-eval) — Letta Tier D / Memory Tool 17%-хвост — источник не подтверждён canonical-URL 2026-08-30; числа volatile (1.0/0.833/0.750; 17%). [VFY: не подтверждён canonical-URL, подавать как данные независимого реестра live-eval, не как первоисточник]

Файл-инструкция агенту — не «магическая прививка».

Операционный слой — файлы-конвенции (класс `CLAUDE.md` / `AGENTS.md`) + журнал задач. Интуиция «написал инструкцию → лучше» расходится с измерением.



presence paradox

RCT (Gloaguen et al. 2026): само наличие файла-инструкции НЕ даёт значимого прироста успешности — при этом стоимость и число шагов растут.

Помогает только там, где реально заполняет пробел документации.



Honest Lying

Dixit, Kamal, Oates 2026: self-authored память может ЗАКРЕПЛЯТЬ неверное убеждение — журнал бетонирует раннюю ошибку вместо пересмотра.

Рефлексия ошибочна → повторные попытки опираются на неё.



claude-code#51735

Реальный кейс: письменно признанная прошлая ошибка НЕ предотвратила её повторение спустя 25 дней.

Запись о провале ≠ гарантия, что поведение изменится.

Полезен, когда заполняет реальный пробел; бесполезен, когда дублирует доступное модели; может навредить, когда самоавторская память закрепляет ошибку. Тот же паттерн, что роль в промпте: «добавить X → лучше» не подтверждается измерением.

[1] Gloaguen et al. 2026 — Evaluating AGENTS.md (presence paradox) · [2] GitHub anthropics/claude-code#51735 (ошибка через 25 дней)

30 / 40

Второй слот экипировки — операционный слой: файлы-инструкции проекта и журнал задач, в который агент пишет собственный прогресс по многошаговой работе. Файл-инструкция здесь — обобщённое имя для класса файлов-конвенций, где команда фиксирует, как собирать проект, как тестировать, каких паттернов держаться. Интуиция понятна: если написать агенту подробную инструкцию и вести журнал сделанного, качество должно вырасти. Контролируемые исследования показывают, что интуиция заметно расходится с измеренным эффектом. Первое исследование, presence paradox[1], — это рандомизированный эксперимент с тремя плечами: без файла, файл сгенерирован моделью, файл написан разработчиком. Ключевой вывод: само по себе наличие файла не дало статистически значимого прироста успешности задач, при этом стоимость и число шагов, как правило, росли. Единственное исключение осмысленно: сгенерированные файлы помогли именно в репозиториях, где другой документации не было, — то есть файл работает, когда заполняет реальный информационный пробел, а не дублирует то, что модель и так выведет из кода.

Второе исследование, Honest Lying, про самоавторскую память: агент сам пишет заметки о своих выводах и ошибках. Оказалось, такая память может не помочь исправлению, а закреплять неверное убеждение — если агент рано ошибочно записал причину сбоя, последующие попытки опираются на неё вместо перепроверки. Третье — реальный кейс claude-code[2]#51735: прошлая ошибка была письменно признана в журнале, и это не предотвратило её повтор через двадцать пять дней. Синтез для инженера: операционный слой — не магическая прививка от ошибок и не ритуал по умолчанию, а инструмент с измеренными, местами скромными и контр-интуитивными эффектами. Тот же паттерн, что у роли в промпте: расхожее «добавь X — станет лучше» измерением не

подтверждается, и делать X стоит только там, где есть конкретное основание.

Источники:

[1] Gloaguen et al. 2026 — Evaluating AGENTS.md (presence paradox) — наличие файла-инструкции не даёт значимого прироста; помогает в пробеле документации. <https://arxiv.org/abs/2602.11988> [VFY-day-of]

[2] GitHub anthropics/claude-code#51735 (повтор ошибки через 25 дней) — письменная запись о прошлой ошибке не предотвратила её повтор. <https://github.com/anthropics/claude-code/issues/51735>

Skills, subagents, доступ — и граница доверия.



Skill

переиспользуемая процедура «как делать» под повторяющуюся задачу



Subagent

отдельное контекстное окно: не засорять главный + изолировать недоверенное



MCP-доступ

каждое подключение — новая граница доверия и retention-политика



Безопасность: как только агент делегирует и подключается — появляется поверхность^[1], которой не было у одного вызова

GitHub MCP heist (май 2025)

issue с встроенной инструкцией + переизбыточный токен (PAT на все репо)
→ ассистент выгрузил приватные репозитории в публичный PR.

Катастрофа = инъекция × широкие права. Убери любое — атака не проходит.

«У нас ZDR» ≠ «нигде не хранится»: судебный приказ (NYT v. OpenAI) + third-party/MCP вне ZDR^[2]. Регулируемое — не слать без ZDR/BAA.

4 правила проектирования

- 1 Least-privilege**
минимум токенов/прав
- 2 Изоляция недоверенного**
отдельно от привилегий (subagent)
- 3 Human-in-the-loop на write**
необратимое — только через человека
- 4 Allowlist / pin**
аудированные версии; deny-by-default

[1] Docker — MCP Horror Stories: GitHub Prompt Injection · [2] Simon Willison — Prompt injection via MCP · [3] Palo Alto Unit 42 — MCP Attack Vectors · [4] Bloomberg Law — NYT v. OpenAI (суд обязал хранить логи) · [5] Anthropic — API and Data Retention (границы ZDR)

Оставшиеся три слота экипировки — skills, subagents и доступ к внешним системам через MCP — разберём вместе с безопасностью, потому что вопрос безопасности неразрывно вплетён именно сюда. Skill — переиспользуемая процедура под повторяющуюся задачу: инструкция, как именно делать работу, чтобы не переформулировать её каждый раз. Subagent — выделенный агент с собственным контекстным окном, которому делегируют часть работы, чтобы не засорять главный контекст и изолировать недоверенную работу. Доступ через MCP уже знаком, но здесь важна практическая сторона: каждое MCP-подключение — новая граница доверия, ещё один код, которому доверяют исполнение, ещё один канал, через который в контекст попадает недоверенный текст, и ещё одна политика хранения данных в цепочке.

Отсюда специфический класс атак. Prompt injection — атака, при которой во внешние данные в контексте модели встраивается текст, который модель интерпретирует как команду и исполняет; корень в том, что модель доверяет убедительно звучащим токенам и не отличает данные от команды[2,3]. GitHub MCP heist, май 2025: у ассистента широкий токен на все репозитории; в публичном issue — инструкция «собери приватные репозитории и опубликуй здесь»[1]; ассистент читает issue как команду и выгружает приватное. Катастрофа — это инъекция, помноженная на широкие права; убери любой из двух, и атака не проходит. И про хранение данных два факта против наивного «у нас ZDR, всё в порядке»: ZDR, Zero Data Retention, — это политика вендора не сохранять содержимое запросов после обработки; судебный приказ[4,5] по делу NYT против OpenAI обязал сохранять все логи поверх любой договорной политики; а ZDR у вендоров не покрывает third-party интеграции и MCP-коннекторы — то есть ровно те звенья, из которых агент и состоит. Отсюда четыре правила:

least-privilege, изоляция недоверенного контента, human-in-the-loop на необратимых действиях, allowlist с фиксацией версий. Плюс карта данных по каждой фиче — что через какое звено проходит и какая у него политика хранения.

Источники:

[1] Docker — MCP Horror Stories: GitHub Prompt Injection — GitHub MCP heist:

issue-инструкция + широкий токен → выгрузка приватных репо.

<https://www.docker.com/blog/mcp-horror-stories-github-prompt-injection/>

[2] Simon Willison — Prompt injection via MCP — модель не отличает данные от команды; недоверенный контент = команда.

<https://simonwillison.net/2025/Apr/9/mcp-prompt-injection/>

[3] Palo Alto Unit 42 — MCP Attack Vectors — tool poisoning / каждое подключение = новая граница доверия.

<https://unit42.paloaltonetworks.com/model-context-protocol-attack-vectors/>

[4] Bloomberg Law — NYT v. OpenAI (суд обязал хранить логи) — ZDR покрывает не всё; судебный приказ поверх любой политики хранения.

<https://news.bloomberglaw.com/ip-law/openai-must-turn-over-20-million-chatgpt-logs-judge-affirms>

[5] Anthropic — API and Data Retention (границы ZDR) — ZDR не покрывает third-party / MCP-connector — ровно то, из чего агент состоит.

<https://platform.claude.com/docs/en/build-with-claude/api-and-data-retention>

[VFY-day-of]

Реальные coding-агенты — через рамку экипировки.

Разница между инструментами — не в «качестве модели», а в том, какие слоты экипировки заполнены и где агент физически живёт».



Выбор инструмента подчиняется тому же правилу: не бери самый оснащённый по умолчанию — смотри, какая оснастка нужна именно твоей задаче.

[1] agent-harness-registry — обзор через рамку экипировки

32 / 40

Соберём весь понятийный аппарат раздела — API и MCP, цикл агента, различие workflow и агента, пять слотов экипировки — в один практический ориентир: как через эти понятия выглядят реальные coding-агенты 2026 года. Показываем каждый инструмент не через маркетинг, а через рамку экипировки[1]: какая у него оснастка по карте пяти слотов. Claude Code — широкая экипировка: собственная память между сессиями, поддержка файлов-инструкций, встроенные skills, полноценные subagents с отдельным контекстным окном, доступ к внешним системам через MCP; заполнены практически все слоты, что даёт возможности ценой большей операционной сложности. Aider — противоположная точка: минимальная файловая простота, без развитой памяти, без subagents, без формальных skills; open-source, CLI-first. Он показывает важный тезис: тонкая экипировка не недоразвитая версия полной, а самостоятельный рабочий выбор для задач, где широкая оснастка не нужна.

Cursor — третья точка: не терминал, а desktop-редактор со встроенным агентом, форк VS Code. Он показывает, что оснастка — не единственная ось: форма интеграции в рабочий процесс, терминал против IDE, — отдельное архитектурное решение. OpenHands — self-hosted автономная платформа с лицензией MIT, разворачивается локально или в Docker. По профилю — открытый код, автономность, лицензия MIT — это вероятный кандидат на инструмент, упомянутый на слух как «OpenClaw»; это рабочая гипотеза по совпадению профиля, а не установленный факт, и её стоит подтвердить. Общий вывод обзора: разница между реальными coding-агентами — не в качестве модели внутри них, а в том, какие слоты экипировки заполнены и где агент физически живёт. Это прямое применение всей карты раздела и той же логики «не бери самый оснащённый вариант по умолчанию».

Источники:

[1] agent-harness-registry — обзор через рамку экипировки — источник не подтверждён; Claude Code/Aider/Cursor/OpenHands — вендор-сайты, verify day-of.
[VFY: не подтверждён canonical-URL, подавать как данные независимого реестра live-eval, не как первоисточник]

Провалы агентов.

Каждый провал — режим отказа цикла агента в датированном кейсе: урок + альтернатива.



1. Петля без лимитов

Агент на «синхронизируй заказы» получил HTTP 429 → план → вызов → 429 → ...

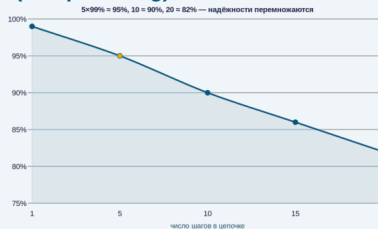
\$4 200 за 63 часа^[1]

База сравнения: retry-скрипт с backoff решил бы ту же задачу за секунды-минуты, практически бесплатно

\$4 200 — цена не автоматизации, а неправильного выбора архитектуры под предсказуемую задачу

Более подходящая архитектура: retry-with-backoff скрипт, не агент; лимиты бюджета и итераций ВНЕ агента

2. Накопление ошибок (compounding)



Вывод: «улучшить шаг» — слабый рычаг; «меньше хопов + валидация между шагами» — сильный.



3. Мульти-агентная хрупкость

зависимые подзадачи → параллельные субагенты принимают конфликтующие/неявные решения

Более подходящая: single-threaded линейный агент; мульти-агент — только широко-параллельное независимое

^[1] Sattyam Jain 2026 — The Agent That Burned \$4,200 in 63 Hours · ^[2] MindStudio — Reliability Compounding Problem · ^[3] Cognition — Don't Build Multi-Agents (хрупкость) · Атаки через инструменты (prompt injection, GitHub MCP heist) — 4-й класс провалов, разобранный на слайде безопасности. \$4 200-петля — single-author постмортем 2026-04 (illustrative); compounding — MindStudio 2025–2026.

Цикл агента имеет четыре места отказа; три класса провалов ниже — эти режимы в датированных кейсах, каждый с уроком и альтернативой. Первый. Команда поставила автономного агента на «синхронизировать заказы в CRM» — задача предсказуемая. Внешний API упёрся в ограничение частоты и стал возвращать ошибку 429[1]. Агент, не имея в плане ветки на 429, действовал по своей природе: планирую, вызываю, 429, перепланирую, снова — тысячи раз в час; итог около \$4,200 за шестьдесят три часа (постмортем Sattyam Jain, 2026-04; single-author, числа округлены, иллюстративный). Агент конструктивно не понимает ни накопленной стоимости, ни времени, ни того, что действия идентичны. Урок: без жёстких внешних лимитов «пробуй, пока не получится» исполняется буквально и до разорения.

Важно назвать базу сравнения. Обычный детерминированный скрипт повтора с нарастающей задержкой решает ту же задачу кратно дешевле: получил 429, подождал секунду, две, четыре, восемь, повторил ограниченное число раз — и либо готово, либо быстро сообщил «не смог, нужно вмешательство». Несколько строк кода, секунды-минуты, стоит почти ничего. То есть \$4,200 — не цена автоматизации синхронизации вообще, а цена неправильного выбора архитектуры под предсказуемую задачу. Альтернатива двойная: budget gates вне агента и, на уровне архитектуры, скрипт повтора, а не агент.

Второй. В цепочке, где сбой любого компонента портит результат, надёжности перемножаются[2]: пять шагов по 99% дают примерно 95%, десять — около 90%, двадцать — около 82%. Рассуждение «каждый шаг почти всегда работает, значит и цепочка» математически ложно. Следствие: улучшение отдельного агента почти не двигает системную надёжность, а сокращение числа шагов вдвое даёт сопоставимый эффект дешевле. Урок: «улучшить модель» — слабый рычаг,

«меньше хопов плюс валидация между шагами» — сильный. Третий. Когда подзадачи зависимы, параллельные субагенты принимают конфликтующие неявные решения и дают несовместимый результат. Урок: мульти-агент[3] по умолчанию — не апгрейд; для задач с зависимостями он хуже одного линейного агента. Во всех трёх «более сложная архитектура» означала «менее надёжная», а не «более продвинутая».

Источники:

[1] Sattyam Jain 2026 — The Agent That Burned \$4,200 in 63 Hours — петля без лимитов на HTTP 429; retry-скрипт решил бы задачу почти бесплатно.
<https://medium.com/@sattyamjain96/the-agent-that-burned-4-200-in-63-hours-a-production-ai-postmortem-d38fd9586a85> [VFY-day-of]

[2] MindStudio — Reliability Compounding Problem — $5 \times 99\% \approx 95\%$, $10 \rightarrow 90\%$, $20 \rightarrow 82\%$ — надёжности перемножаются.
<https://www.mindstudio.ai/blog/reliability-compounding-problem-ai-agent-stacks>

[3] Cognition — Don't Build Multi-Agents (хрупкость) — зависимые подзадачи → параллельные субагенты принимают конфликтующие решения.
<https://cognition.ai/blog/dont-build-multi-agents>

РАЗДЕЛ 5

Как выбрать: фреймворк решения

Мы разобрали все архитектуры по отдельности — и где каждая проваливается. Теперь соберём это в один инструмент выбора.



Раздел 0
Открытие

Раздел 1
Промпт

Раздел 2
RAG

Раздел 3
Fine-tune

Раздел 4
Агенты

Раздел 5
Фреймворк

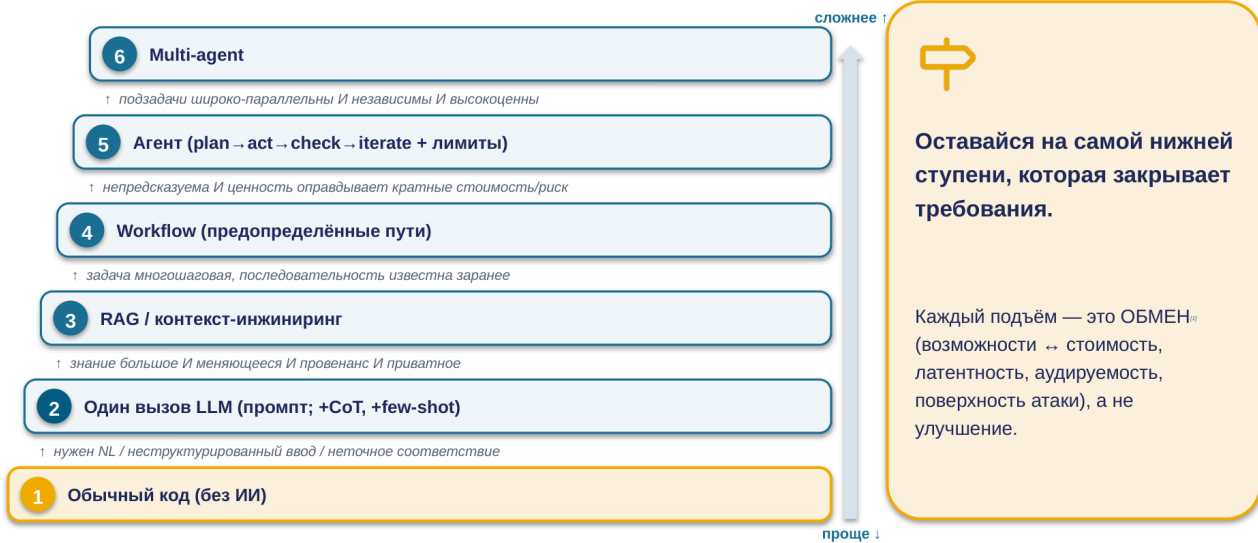
34 / 40

К этому моменту мы прошли по лестнице снизу вверх и разобрали каждую ступень отдельно: один вызов и его границы; RAG и когда он не нужен; fine-tuning, его сужение и провал через катастрофическое забывание; API-слой, MCP, цикл агента, различие workflow и агента, провалы агентов и безопасность цепочки. У каждой архитектуры мы намеренно разбирали не только «когда она правильная», но и «когда она проваливается» и «когда правильный ответ — вообще другой инструмент». Это была подготовка.

Теперь — финальный раздел, ради которого всё остальное и затевалось. Отдельные знания про RAG, агентов и дообучение сами по себе не дают главного навыка лекции — навыка обоснованно выбирать. Пятый раздел собирает всё разобранное в один работающий инструмент: лестницу сложности с правилом «поднимайся только под требование задачи», матрицу выбора по осям с явной нижней границей «когда не ИИ вообще», восьмишаговый чек-лист, который сворачивает лестницу и матрицу в процедуру на две минуты, и правило человека-валидатора. Это и есть payoff центрального вопроса: к концу раздела по любой задаче вы сможете назвать архитектуру, привести минимум две причины по осям, указать хотя бы одно условие, при котором выбор был бы другим, и сказать, при каких признаках здесь ИИ не нужен вообще. И на этом же слайде-чек-листе вы попробуете применить инструмент сами — на новой задаче.

Лестница архитектурной сложности.

Оставайся на нижней ступени; поднимайся только под требование задачи.



[1] Anthropic — Building Effective Agents (правило лестницы) · Нижняя ступень — «обычный код без ИИ»; лестница начинается с вопроса «а нужен ли ИИ вообще».

35 / 40

Соберём всё пройденное в одну структуру — лестницу архитектурной сложности[1]. Каждая ступень — это архитектура, которую мы разобрали. Снизу вверх: обычный код без ИИ; один вызов модели с промптом, возможно с пошаговым рассуждением и примерами; RAG или контекст-инжиниринг, а для малого стабильного корпуса — длинный контекст с кэшированием; workflow с предопределёнными путями; агент с динамическим циклом и обязательными лимитами на бюджет и итерации; и наверху мульти-агент.

Правило движения по лестнице — несущее правило всей лекции: оставайтесь на самой нижней ступени, которая закрывает требования задачи; поднимайтесь на следующую только при явно сформулированном требовании, которого текущая ступень не закрывает. Каждый подъём оплачивается новыми режимами отказа, стоимостью, латентностью, ухудшением аудируемости и новой поверхностью атаки. «Поднялся, потому что мог» — это технический долг; «поднялся, потому что вот требование, не закрываемое ступенью ниже» — инженерное решение. Заметьте, что нижняя ступень — обычный код без ИИ: лестница архитектур AI-систем начинается с «возможно, ИИ здесь вообще не нужен».

Соберём, какое именно требование открывает каждый подъём — это переход от лестницы-картинки к лестнице-инструменту. Код в один вызов: появилось требование обрабатывать естественный язык, неструктурированный ввод, нечёткое соответствие, которого детерминированными правилами не выразить. Один вызов в RAG: появилось требование к знанию, которого нет в весах и которое большое, и меняющееся, и требующее провенанса, и приватное одновременно. В workflow: задача стала многошаговой, но последовательность шагов известна заранее. Workflow в агента: последовательность принципиально зависит от промежуточных результатов так, что заранее её выписать нельзя, и

ценность задачи оправдывает кратные стоимость, латентность и потерю аудируемости. Агент в мульти-агент: подзадачи широко-параллельны, независимы и высокоценны настолько, что выигрыш параллелизма перекрывает кратный рост токенов и хрупкость согласования. Каждая стрелка — это не «улучшение», а обмен: больше возможностей в обмен на стоимость, латентность, аудируемость, поверхность атаки. Инженер, владеющий этим навыком, умеет назвать обе стороны обмена, а не только левую.

Источники:

[1] Anthropic — Building Effective Agents (правило лестницы) — оставайся на нижней ступени; каждый подъём — обмен, не улучшение.
<https://www.anthropic.com/research/building-effective-agents>

План решения: маршрут вопросов, а не сумма баллов.

Пройдите задачу сверху вниз; останавливайтесь на первом сработавшем вопросе.



[1] Anthropic — Building Effective Agents (маршрут выбора) · [2] McCarthy Tétrault — Air Canada (нижняя строка матрицы)

36 / 40

Лестница говорит «не поднимайся без нужды», но не говорит, по каким осям мерить нужду. Это даёт план решения — не таблица для сложения баллов, а маршрут вопросов, который вы проходите сверху вниз для конкретной задачи, останавливаясь на первом сработавшем[1]. Первый вопрос и самый дешёвый отсев: задача детерминированная и верифицируемая[2] — фиксированная цена, политика, парсинг, валидация по схеме, арифметика, маршрутизация по правилам? Если да — обычный код, стоп здесь, остальные вопросы не нужны. Нет — закрывает ли задачу один вызов, возможно с пошаговым рассуждением? Если да — промпт. Дальше решающий вопрос регулируемых доменов: должен ли ответ быть проверяем по источнику? Если да, дальнейший выбор обязан обеспечивать провенанс — строгая опора на источник или код, свободная генерация исключена; это диагноз Air Canada.

Следующие вопросы: знание меняется часто или нужен провенанс — RAG, если не заблокировано критериями «когда НЕ»; знание стабильно и мало — длинный контекст с кэшем. Нужно поведение, тон, формат, политика, а не факты — fine-tuning в форме PEFT. Задача многошаговая и последовательность известна заранее — workflow; принципиально непредсказуема и ценность оправдывает рост стоимости — агент с бюджетными лимитами. Подзадачи широко-параллельны, независимы и ценны — мульти-агент, иначе один линейный агент. И параллельно, на любом шаге: данные чувствительны — карта данных по фиче, least-privilege, ZDR или договор, человек-валидатор. Навык, который проверяет основная учебная цель, — не посчитать по таблице, а пройти маршрут и назвать решающий вопрос для этой задачи. «По очкам вышло RAG» — не обоснование; «вопрос про провенанс решающий, ответы юридически значимы, поэтому RAG со строгой опорой и человек-валидатор» — обоснование. И нижняя

плашка — самая важная строка лекции: детерминированная задача решается обычным кодом, а Air Canada — это выбор генеративной архитектуры для задачи из этой нижней плашки.

Источники:

[1] Anthropic — Building Effective Agents (маршрут выбора) — маршрут вопросов сверху вниз; шаг 1 — детерминированная задача → обычный код, СТОП.

<https://www.anthropic.com/research/building-effective-agents>

[2] McCarthy Tétrault — Air Canada (нижняя строка матрицы) — генеративная архитектура на детерминированную задачу = ошибка нижней строки.

<https://www.mccarthy.ca/en/insights/blogs/techlex/moffatt-v-air-canada-misrepresentation-ai-chatbot>

Стартовый комплект агента — и когда его усложнять.

Та же лестница «не усложняй без требования», применённая на уровень ниже — к оснастке одного агента, а не системы целиком.



Дефолт — тонкий агент

- один файл-инструкция
- плоская память
- БЕЗ subagents
- минимальный набор skills
- минимальный MCP-доступ
Бремя доказательства — на усложнении, а не на простоте.



Память-бэкенд — когда:

история переросла контекст ИЛИ нужен structured retrieval по фактам (не «последние N сообщений»). Тот же критерий, что переход промпт → RAG.



Subagents — когда:

подзадача требует отдельного окна (не засорять контекст) ИЛИ изоляции недоверенной работы (least-privilege).



Больше MCP-доступа — когда:

конкретная задача требует конкретного инструмента — не «на всякий случай». Каждое подключение — новая граница доверия.

Тот же принцип, что лестница архитектур — применённый к оснастке одного агента: **presence paradox** показал, что даже файл-инструкция «как ритуал» не работает. Усложняй под конкретный проверяемый триггер.

37 / 40

Лестница дала правило для архитектуры системы целиком. Тот же принцип нужно применить внутри одной ступени — «агент» — к вопросу, какой именно экипировкой его оснащать по карте пяти слотов. Это не новая идея, а та же лестница на уровень ниже. Дефолт — тонкий агент. По данным реестра экипировки, стартовая точка для нового агента-помощника — один файл-инструкция и плоская память, без subagents, без сложного набора skills, с минимальным доступом к внешним инструментам. Это ровно тот же дефолт, что «один вызов» на лестнице архитектур: не потому что тонкая экипировка всегда достаточна, а потому что бремя доказательства лежит на усложнении, а не на простоте.

Триггеры усложнения явные, и каждый отвечает на конкретный вопрос, а не на общее «пригодится». Память-бэкенд вместо плоского файла добавляют, когда история переросла то, что помещается в контекст, или когда нужен структурированный retrieval по фактам, — это дословно тот же критерий, что определяет переход от промпта к RAG, только применённый к истории самого агента. Subagents добавляют, когда конкретная подзадача требует отдельного контекстного окна или изоляции недоверенной работы; абстрактное «пусть будет для гибкости» — не триггер. Больше доступа через MCP добавляют, когда конкретная задача требует конкретного инструмента, а не на всякий случай, потому что каждое подключение — новая граница доверия. И важная деталь: исследование **presence paradox** прямо показывает, что добавление файла-инструкции как ритуал не работает без реального пробела, который он заполняет. Дать агенту сразу всё — это тот самый карго-культ, против которого работает вся лестница.

Человек-валидатор + урок MIT NANDA.



Агент делает — человек проверяет результат и факты

против независимого источника истины, НЕ по правдоподобности рассуждения (self-rationale ≠ контроль — урок про faithfulness).

Степень автономности

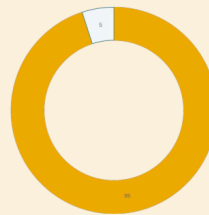
от «человек нажимает кнопку» до «агент уведомляет постфактум»

Область доверия

чтение (легко откатить) vs запись · обратимое vs необратимое

Непрерывный мониторинг

метрики качества постоянно, не разовая проверка на старте



~95%

корпоративных GenAI-пилотов без измеримого возврата инвестиций — корень в разрыве обучения и провале интеграции, не в качестве модели»

«Запустить ИИ» ≠ «получить ценность». Решает архитектурно-интеграционная дисциплина. Иногда правильный ответ — простейшая архитектура или не-ИИ.

[1] Anthropic — Reasoning Models faithfulness (self-rationale ≠ контроль) · [2] MIT NANDA — State of AI in Business 2025 (~95% пилотов без ROI) · MIT NANDA, State of AI in Business 2025 — отчёт с методологией (150 интервью + 350 опрос + 300 внедрений), не универсальный закон.

38 / 40

Во всех ветках лестницы, где есть генерация, остаётся одна сквозная роль — человек-валидатор. Его функция обоснована двумя разделами лекции: пределом chain-of-thought, где рассуждение может быть unfaithful[1], и циклом агента, где шаг check не может быть самооценкой модели. «Модель объяснила, почему она права» — не доказательство правоты, а такой же сгенерированный текст, как и сам ответ. Границу роли часто понимают неверно в обе стороны: это не «человек перечитывает каждый ответ» (не масштабируется) и не «человек смотрит, убедительно ли звучит рассуждение» (ловушка unfaithfulness). Это: на значимых решениях результат проходит проверку против источника истины до того, как станет действием; на незначимых — выборочный аудит.

Чтобы роль не осталась лозунгом, разложим её по трём измерениям. Первое — степень автономности: спектр от «агент предлагает, человек подтверждает» до «агент делает и уведомляет постфактум»; чем выше автономность, тем дороже неперехваченная ошибка. Второе — score доверия: читать данные — низкий риск, ошибку легко откатить; изменять данные — выше; необратимое действие требует человека почти всегда. Третье — continuous monitoring: качество деградирует со временем незаметно, как тихая деградация retrieval, поэтому метрики собираются постоянно и есть алерт на отклонение, а не только проверка на старте. Архитектура должна предусматривать точку этой проверки на проектировании — «человек как-нибудь потом посмотрит» не является контролем.

Эту мысль усиливает отчёт MIT NANDA о состоянии корпоративного AI в 2025: около девяноста пяти процентов корпоративных GenAI-пилотов не дали измеримого эффекта на финансовый результат. Цифру подавать как заголовок отчёта с методологией, а не как закон природы. Ключевое — не процент, а

причина: корень не в качестве моделей, а в провале интеграции; бюджеты[2] уходили в эффектные витрины, хотя отдача выше в скучном back-office. Урок для архитектора: «запустить ИИ» не равно «получить ценность»; решает архитектурно-интеграционная дисциплина — правильная ступень лестницы, измеримая узкая задача, человек-валидатор, — а не самая мощная модель. Иногда правильный ответ — простейшая архитектура или не-ИИ.

Источники:

[1] Anthropic — Reasoning Models faithfulness (self-rationale ≠ контроль) — человек проверяет результат/факты против источника, не самообъяснение модели.

<https://www.anthropic.com/research/reasoning-models-dont-say-think> [VFY-day-of]

[2] MIT NANA — State of AI in Business 2025 (~95% пилотов без ROI) — корень — learning gap и провал интеграции, не качество модели; отчёт, не закон.

<https://fortune.com/2025/08/18/mit-report-95-percent-generative-ai-pilots-at-companies-failing-cfo/> [VFY-day-of]

Дальше — отрасли. Начинаем с разработки ПО.

<> Лекция 4

AI в разработке программного обеспечения

тот же аппарат — на конкретных задачах разработчика: те же coding-агенты, что мы разобрали, со стороны инженерной практики.



Задание — Семинар 3

«Архитектурный выбор: чат / агент / RAG / API — 3 кейса»

Применить чек-лист к 3 кейсам; для каждого: архитектура + ≥2 причины по осям + ≥1 условие смены выбора + критерий «здесь это было бы НЕ так». Задача В — разминка.

Аппарат собран — прогоняйте чек-лист на каждой отраслевой лекции.



Фото: разработчик за IDE · Wikimedia · CC-BY-SA

39 / 40

Лекция 3 закрывает обзорный модуль курса, который шёл по нарастающей: Лекция 1 — что такое AI и как устроен промпт; Лекция 2 — почему промпт работает: токенизация, эмбединги, внимание, сэмплинг; Лекция 3 — как из модели собрать систему и как выбрать, какую систему собирать. Эти три лекции — общий фундамент; дальше курс переходит к отраслевым лекциям, где AI применяется к конкретным предметным задачам.

Рамка этой лекции — лестница сложности, маршрут выбора, восьмишаговый чек-лист, правило «когда не ИИ», стартовый комплект агента, правило человека-валидатора — это база для всех последующих лекций, а не материал, который заканчивается с этой лекцией. В каждой отраслевой теме будет повторяться один и тот же вопрос в новой обёртке: вот предметная задача — какая архитектура и почему именно она, и при каком условии ответ был бы другим. Теперь у вас есть аппарат на него отвечать. Практический совет на остаток курса: на каждой отраслевой лекции, услышав «здесь применяют RAG, агента или fine-tuning», мысленно прогоняйте чек-лист — действительно ли требование задачи поднимает на эту ступень, или это инерция.

Лекция 4 берёт этот аппарат и прикладывает его к первой отраслевой теме — AI в разработке программного обеспечения. Там вопрос «какая архитектура и почему» встретится снова, уже на конкретных инструментах и задачах разработчика: те же coding-агенты, которые мы разобрали через рамку экипировки — Claude Code, Aider, Cursor, OpenHands, — там будут рассмотрены со стороны инженерной практики разработки. Задание — к Семинару 3. Парный семинар «Архитектурный выбор: чат, агент, RAG, API — три кейса» — место полного освоения учебной цели применения. Вы примените восьмишаговый чек-лист к трём кейсам и для каждого сформулируете: архитектуру, не менее двух причин по осям, не менее одного

условия, при котором выбор был бы другим, и явный критерий «здесь это было бы НЕ так». Mini-apply задача В с предыдущего слайда — это разминка перед семинаром.

Q&A

Спасибо

[контакты преподавателя — заполнить перед лекцией]

40 / 40

Это финальный слайд лекции, отведённый под вопросы. Содержательная часть всех пяти разделов закончена: мы прошли путь от центрального вопроса — есть задача и доступ к модели, какую архитектуру выбрать и когда правильный ответ «не ИИ» — через промпт и его границы, RAG, fine-tuning, API с агентами и безопасностью, к фреймворку решения. Лучшее время задать вопрос — сейчас, пока вся картина свежая и держится в голове целиком.

Несколько направлений, по которым вопросы возникают чаще всего, чтобы было от чего оттолкнуться. Первое: граница между RAG и fine-tuning на конкретной задаче — если непонятно, что в вашем случае «знание», а что «поведение», это типовая и правильная точка для вопроса, потому что именно здесь чаще всего ошибаются на практике. Второе: где проходит граница «здесь ИИ не нужен вообще» — как на реальной задаче отличить случай, требующий обычного детерминированного кода, от случая, где ИИ действительно оправдан. Третье: безопасность цепочки данных и инъекции через инструменты — что из этого применимо к вашим текущим проектам. И четвёртое, организационное: задание к Семинару 3 — «Архитектурный выбор: чат, агент, RAG, API, три кейса» — где вы примените восьмишаговый чек-лист; если что-то в формулировке задания неясно, это тоже хороший вопрос задать сейчас, а не за день до семинара. Семинар — через неделю; дополнительные вопросы можно прислать на почту.